



DOSW COMPANY



ESCUELA COLOMBIANA DE INGENIERIA

SEMANA 10

DOSW 1

EN QUE QUEDAMOS LA CLASE PASADA?





TU RESTAURANTE YA TIENE LO MINIMO PARA FUNCIONAR...

Ya construida nuestra aplicacion con los primeros cimientos, es necesario que tener en cuenta que nos falta un componente bastante importante

LA SEGURIDAD DE NUESTRO RESTAURANTE

No todo el mundo puede hacer todo en el restaurante, el cliente puede ver la carta y hacer pedidos, pero no puede actualizar el estado del pedido, un mesero no puede ordenar



TODOS EN SUS COMPUTADORES



Van a ejecutar su aplicacion con spring y tendran su aplicacion corriendo en localhost:8080

Van a mostrar en su navegador su swagger corriendo, vamos a revisar si cumplieron con la asignacion para la clase de hoy

Ejecuten una funcionalidad de creacion, consulta y borrado para verificar que funcione

La condicion es que se vea reflejada en su BD, ejecuten algo relacional y no relacional



DOSW COMPANY

SEGURIDAD EN SERVICIOS

PARA NUESTRO ECOSISTEMA UTILIZAREMOS ...



Es un framework de seguridad potente y personalizable que funciona como el estándar de facto para la autenticación y el control de acceso en aplicaciones basadas en Spring. Su propósito principal es proteger las aplicaciones Java frente a vulnerabilidades y accesos no autorizados



AUTHENTICATION

Es el proceso de verificar la identidad del usuario o cliente que intenta acceder a la

API
Responde a la pregunta: "¿Quién eres
tú?"



COMO SE APLICA EN UNA API REST

Usualmente, el cliente (por ejemplo, una app o un frontend) envía credenciales (usuario/contraseña, tokens, claves de API, etc.) al servidor.

Si las credenciales son válidas, la API confía en que quien hace la petición es quien dice ser.

HTTP BASIC

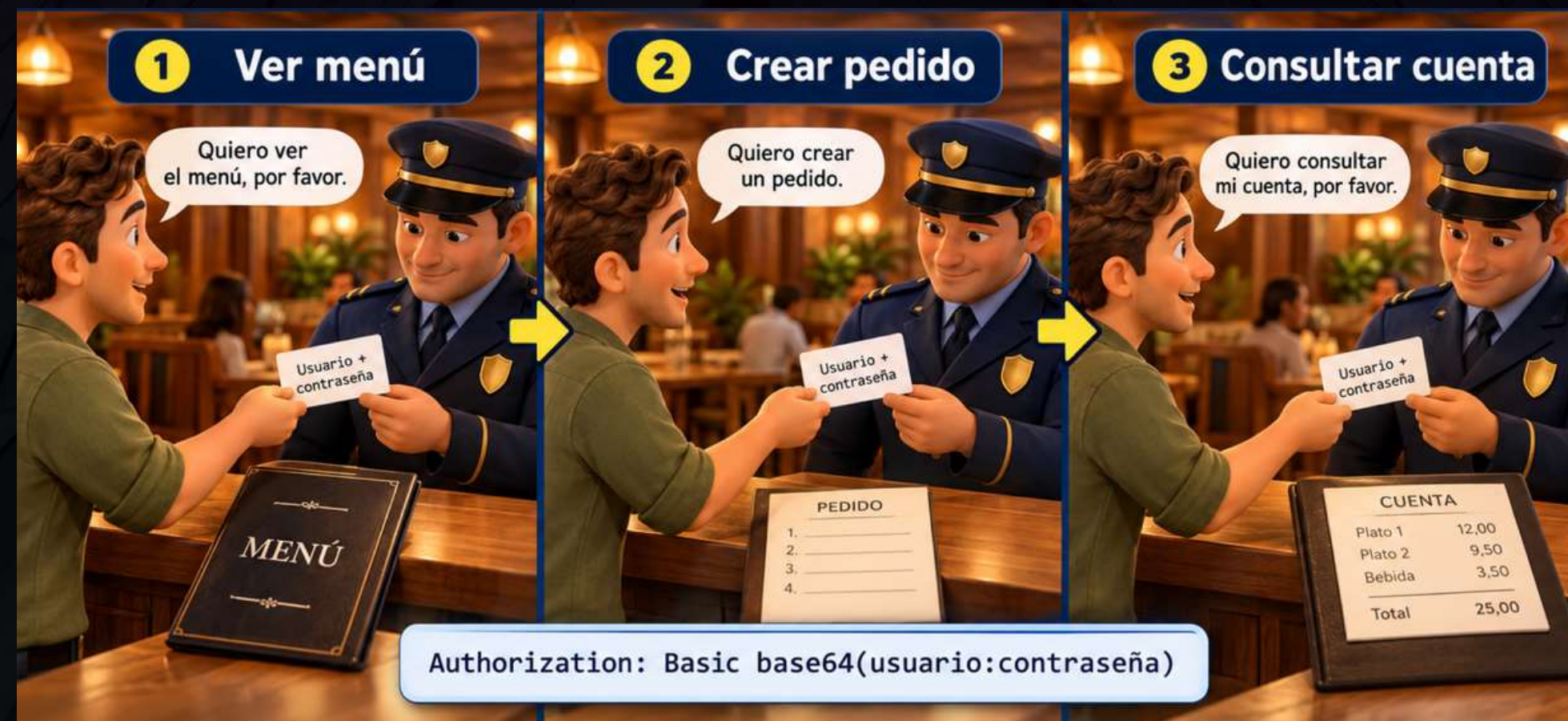
Es el mecanismo mas simple de autentificacion HTTP
Se envía usuario y contraseña codificados en base64 en el header

Authorization
No es cifrado, solo
codificación

EJEMPLO

Peticion
HTTP

```
GET /api/users/me HTTP/1.1  
Host: api.ejemplo.com  
Authorization: Basic YW5kcmVzOjEyMzQ1
```



Codificacio
n

YW5kcmVzOjEyMzQ1

andres:12345

OAUTH 2

Es un protocolo de autorización que permite a una app acceder a recursos en nombre de un usuario sin conocer su contraseña

Permite autenticarse usando un proveedor externo como Google, Facebook, etc, y obtener un token temporal

FLUJO BASICO

- 1.El usuario inicia sesión con el proveedor de identidad (Google).
- 2.Google le da un authorization code a la app.
- 3.La app canjea ese código por un access token.
- 4.La app usa ese token para acceder a la API en nombre del usuario.



```
GET /api/profile
Authorization: Bearer ya29.a0AVvZVsrT...
```

JWT





JWT

JWT (JSON Web Token) es un token seguro y firmado que se usa para autenticación sin estado (stateless).

Sirve para evitar mantener sesiones en el servidor: toda la info de autenticación está dentro del token.

ESTRUCTURA

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
eyJ1c2VySWQiOiJhbmRyZXMiLCJyb2xlIjoieWRtaW4ifQ.  
jzTiBqg1nXqQ6vksQUhGf2pgQLMQEazCgZAVHbPBm9A
```

Tiene 3 partes separadas por

puntos
Header.Payload.Signature
e

```
{ "alg": "HS256", "typ": "JWT" }
```

Header donde esta el tipo de token y algoritmo de firma

```
{ "userId": "andres", "role": "admin" }
```

payload con datos del usuario no sensibles

```
HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)
```

Firma digital para validar integridad



SPRING SECURITY Y JWT

El cliente obtiene un token de acceso al autenticarse por ejemplo usando /login, y luego lo incluye en cada petición, por lo general tiene un tiempo de vida (TTL)

EJEMPLO

Generar un JWT

```
String token = Jwts.builder()
    .setSubject("andres")
    .claim("role", "ADMIN")
    .setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + 3600000)) // 1 hora
    .signWith(SignatureAlgorithm.HS256, "mi_clave_secreta")
    .compact();
```

Validar JWT

```
String token = header.replace("Bearer ", "");
Claims claims = Jwts.parser()
    .setSigningKey("mi_clave_secreta")
    .parseClaimsJws(token)
    .getBody();
String username = claims.getSubject();
String role = claims.get("role", String.class);
```

AUTHORIZATION

Una vez autenticado el usuario, la autorización define qué puede hacer dentro del sistema.
Responde a la pregunta: "¿Qué tienes permitido hacer?"



COMO SE APLICA EN UNA API REST

Se evalúan los roles, permisos o scopes del usuario autenticado.
La API permite o niega el acceso a ciertos recursos o acciones.



SECURING URLS - PERMISOS POR ROLES

Permite restringir el acceso a ciertas partes de la aplicación según el rol del usuario (admin, user, etc.).

COMO SE APLICA EN UNA API REST

Spring Security te permite definir reglas de acceso directamente en tus métodos o clases, usando anotaciones como:

```
@PreAuthorize("hasAnyRole('ESTUDIANTE', 'PROFESOR')")
@GetMapping("/materiales")
public List<Material> obtenerMateriales() {
    return materialService.listar();
}
```

Evaluar una expresion antes de
ejecutar el metodo

```
@PreAuthorize("hasRole('ADMIN') or hasRole('PROFESOR')")
```

Permite expresiones logicas

```
@PreAuthorize("#userId == authentication.principal.id or hasRole('ADMIN')")
```

Permite condiciones dinamicas



SECURING URLS

Si tu API usa JWT, puedes incluir roles o permisos directamente en el token.

```
{  
  "sub": "andres",  
  "roles": ["ESTUDIANTE", "PROFESOR"],  
  "permissions": ["READ_CURSO", "CREATE_TAREA"]  
}
```

Luego pueden usar el filtro de autenticacion en Spring para mapear esos claims a GrantedAuthorities

```
claims.get("roles", List.class)  
  .forEach(role -> authorities.add(new SimpleGrantedAuthority("ROLE_" + role)));
```

Y con eso pueden
usar

```
@PreAuthorize("hasRole('PROFESOR')")
```



INTEGRITY

La integridad garantiza que los datos no se han modificado o alterado durante la transmisión o

almacenamiento.
Responde a la pregunta: "El mensaje o dato llegó exactamente como fue enviado?"



COMO SE APLICA EN UNA API REST

Usando HTTPS (SSL/TLS) para cifrar la comunicación y evitar que los datos se alteren en tránsito.

Usando firmas digitales o HMAC (Hash-based Message Authentication Code) para validar que el cuerpo de la petición o respuesta no fue modificado.

En algunos sistemas, los tokens JWT incluyen una firma digital (firma HMAC o RSA) que garantiza que el contenido del token no fue alterado.



SSL

SSL/TLS cifra la comunicación entre cliente y servidor para:
Evitar que alguien intercepte los datos (man-in-the-middle).
Garantizar confidencialidad e integridad.

COMO SE APLICA EN UNA API REST

Siempre se debe usar https:// en lugar de http://.
Se configura un certificado digital (de Let's Encrypt o CA).

```
https://api.payu.com/v1/payments
```



CORS

Es un mecanismo de seguridad del navegador que controla que recursos pueden ser solicitados desde un dominio diferente al del servidor original, impidiendo que una pagina web cargada en un dominio haga peticiones a otro dominio a menos que el servidor lo permita, es fundamental en API REST modernas sobre todo en comunicacion front - back

COMO FUNCIONA

Cuando haces una peticion desde un dominio distinto, el navegador agrega cabeceras CORS especiales

```
fetch("https://api.backend.com/users", {  
  method: "GET",  
  headers: { "Authorization": "Bearer token" }  
});
```

El navegador no envia la peticion directamente, primero hace una pre solicitud

```
OPTIONS /users HTTP/1.1  
Origin: https://frontend.com  
Access-Control-Request-Method: GET  
Access-Control-Request-Headers: Authorization
```

```
HTTP/1.1 200 OK  
Access-Control-Allow-Origin: https://frontend.com  
Access-Control-Allow-Methods: GET, POST, OPTIONS  
Access-Control-Allow-Headers: Authorization, Content-Type
```

El servidor responde con el access control, si el navegador recibe esas cabeceras permite la peticion si no la bloquea



EJEMPLO DE ERROR CORS

```
Access to fetch at 'https://api.backend.com/users' from origin 'https://frontend.com'
has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present.
```

COMO HABILITARLO

Con springboot a nivel global

```
@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/**")
                    .allowedOrigins("https://frontend.com")
                    .allowedMethods("GET", "POST", "PUT", "DELETE")
                    .allowedHeaders("*")
                    .allowCredentials(true);
            }
        };
    }
}
```

A nivel de controlador

```
@CrossOrigin(origins = "https://frontend.com")
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping
    public List<User> getUsers() { ... }
}
```



SEGURIDAD EN SERVICIOS DE SOFTWARE

La seguridad de las APIs es el proceso de proteger las APIs de los ataques. Al igual que las aplicaciones, las redes y los servidores pueden ser objeto de ataques, las APIs pueden sufrir de diversas amenazas

La mayoría de las aplicaciones web modernas dependen de las APIs para funcionar, y las APIs introducen un riesgo adicional para una aplicación al permitir que terceros accedan a la misma.

Una comparación es una empresa que abre su oficina al público: tener más personas en las instalaciones, algunas de las cuales pueden ser desconocidas para los empleados de la empresa, introduce un mayor riesgo.

De manera similar, una API permite a personas externas usar un programa, lo que introduce más riesgo para la infraestructura del servicio de la API.



RIESGOS DE SEGURIDAD

Cuando un atacante envia datos especialmente elaborados a un objetivo, datos que se aprovechan de defectos en la construccion del objetivo - Se presenta un aprovechamiento a las vulnerabilidades.

OWASP mantiene el listado de las 10 vulnerabilidades mas criticas de las API

QUE ES OWASP TOP 10?

- Es una guía de concienciación creada por la fundación [OWASP](#) para ayudar a desarrolladores y empresas a escribir código más seguro.
- Resume las vulnerabilidades más peligrosas basándose en datos de seguridad y consenso global.

OWASP TOP 10



Broken Access Control – Los usuarios pueden acceder a datos o funciones fuera de sus permisos

Cryptographic Failures – Protección inadecuada de datos confidenciales en tránsito o en reposo (contraseñas o tarjetas de crédito)

Injection – Ocurre cuando se envían datos no confiables a un intérprete como SQL o comandos del sistema y permite ejecutar comandos no autorizados

Insecure Design – Defectos en la arquitectura del software antes de escribir código – Perfiles de seguridad desde el inicio

Security Misconfiguration – Falta de hardening en los sistemas, como mantener configuraciones por defecto, mensajes de error muy detallados o servicios innecesarios activos



OWASP TOP 10



Vulnerable and Outdated Components - Uso de software, bibliotecas o dependencias antiguas que tienen vulnerabilidades conocidas y ya reportadas.

Identification and Authentication Failures - Debilidades en la confirmación de la identidad del usuario, lo que facilita ataques de fuerza bruta o el robo de sesiones.

Software and Data Integrity Failures - Código e infraestructura que no protegen contra violaciones de integridad (por ejemplo, complementos o actualizaciones de fuentes no verificadas).

Security Logging and Monitoring Failures - Incapacidad para detectar, registrar y responder a brechas de seguridad activas en tiempo real

Server-Side Request Forgery - SSRF (Falsificación de solicitudes del lado del servidor) - Ocurre cuando una aplicación web busca un recurso remoto sin validar la URL introducida por el usuario, permitiendo atacar sistemas internos

CUALES SON LOS RIESGOS DE SEGURIDAD?

La gran mayoría son ataques basados en autenticación, como los clientes necesitan autenticarse antes de realizar las solicitudes a una API para que el servidor no acepte solicitudes ilegítimas o desconocidas, existen ataques relacionados con la obtención de credenciales de manera ilícita, robar una clave de la API o interceptar y utilizar un token de autenticación



CUALES SON LOS RIESGOS DE SEGURIDAD?

la autorización determina el nivel de acceso que tiene cada usuario. Si la autorización no se gestiona con cuidado, un cliente de la API podría tener acceso a datos que no deberían estar a su disposición, lo que aumenta la posibilidad de que se produzca una fuga de datos.



Demasiadas solicitudes dirigidas a una API pueden ralentizar o detener el servicio para otros clientes. Algunos atacantes envían una cantidad enorme de solicitudes a una API a propósito en un ataque de denegación de servicio (DoS) o de denegación de servicio distribuido (DDoS).



CONOZCAMOS ALGUNOS....

CSRF (CROSS-SITE REQUEST FORGERY)

Un atacante intenta ejecutar acciones en nombre de un usuario autenticado sin su consentimiento (por ejemplo, haciendo clic en un link malicioso).



LAS APIS STATELESS (JWT) NO SUELEN SER VULNERABLES A CSRF, PERO EN APPS WEB TRADICIONALES SE MANEJA CON SPRING SECURITY

XSS (CROSS-SITE SCRIPTING)

Ocurre cuando datos controlados por un atacante terminan ejecutándose como código en el navegador de otro usuario. En una aplicación con Spring Boot y React, la prevención depende tanto del backend como de cómo el frontend presenta esos datos.



PREVENCION

Escapar contenido HTML (<script> → <script>).
Usar librerías seguras para plantillas.
Configurar encabezados HTTP:

```
Content-Security-Policy: default-src 'self'
```

CLICKJACKING

Es engañar al usuario para que haga clic en una acción real de tu aplicación creyendo que está pulsando otra cosa.

Por ejemplo:

1. Un atacante muestra una página con el botón “Reclama tu premio”.
2. Encima coloca un iframe transparente con tu aplicación.
3. El usuario cree que reclama el premio, pero realmente pulsa “Confirmar operación” en tu aplicación, donde tiene una sesión activa.

PREVENCIÓN

Agregar encabezados HTTP

como

```
X-Frame-Options: DENY
```

```
Content-Security-Policy: frame-ancestors 'none';
```



YA TENEMOS NUESTRA API BASICA ...



Ya sabemos como plasmar las funcionalidades y como organizarlas

Ya sabemos de donde obtenemos la informacion y como la podemos almacenar



Ya sabemos como hacerla segura y que todos cumplan con sus responsabilidades

PERO NOS FALTA ALGO IMPORTANTE...

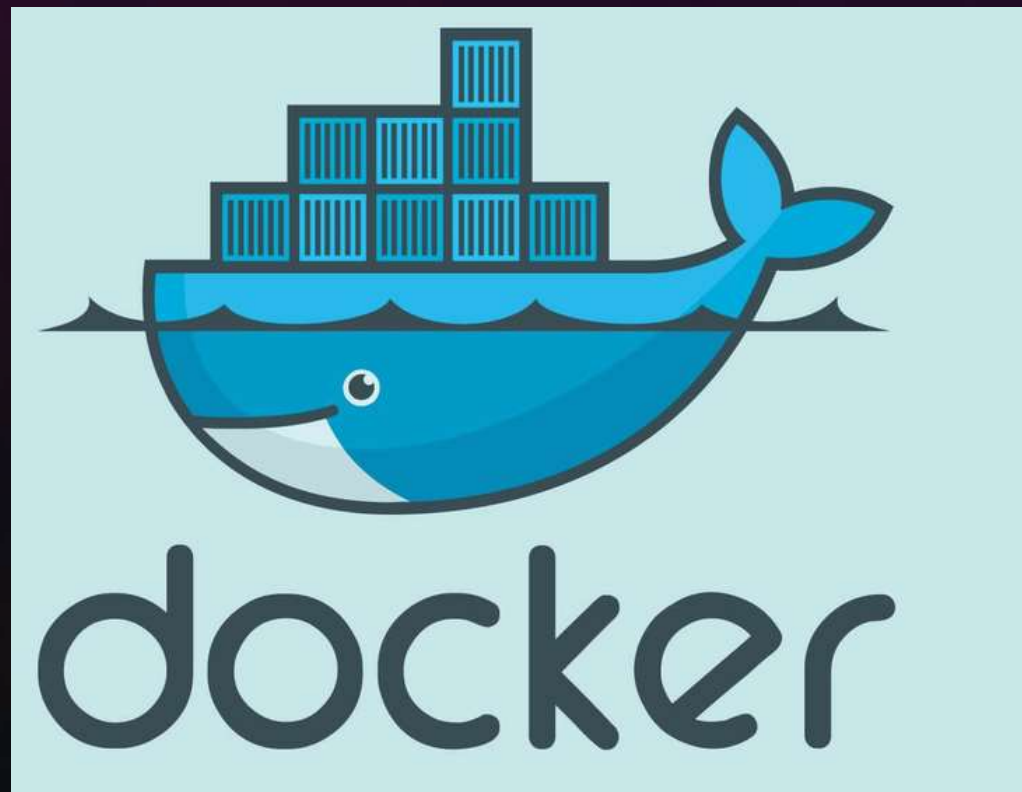


Imagina que tu restaurante tiene éxito y quieres abrir otra sede. No basta con enviar las recetas: necesitas los utensilios, los ingredientes y los equipos adecuados. Si cada sede utiliza lo que encuentra, los platos pueden salir distintos.



DOSW COMPANY

DOCKERIZACION Y USO DE CI/CD



Docker permite empaquetar tu API con las dependencias y la configuración base que necesita para ejecutarse, como un kit de cocina que puedes reproducir en distintas sedes.

“Si mi plato queda bien en mi cocina, quiero poder prepararlo igual en la cocina de pruebas y en la del restaurante que atiende al público”.

Docker es una plataforma para crear, empaquetar y ejecutar aplicaciones dentro de contenedores.



Un contenedor es como una “caja ligera” que incluye:

- La aplicación.
- Sus dependencias.
- Librerías y configuración necesarias.



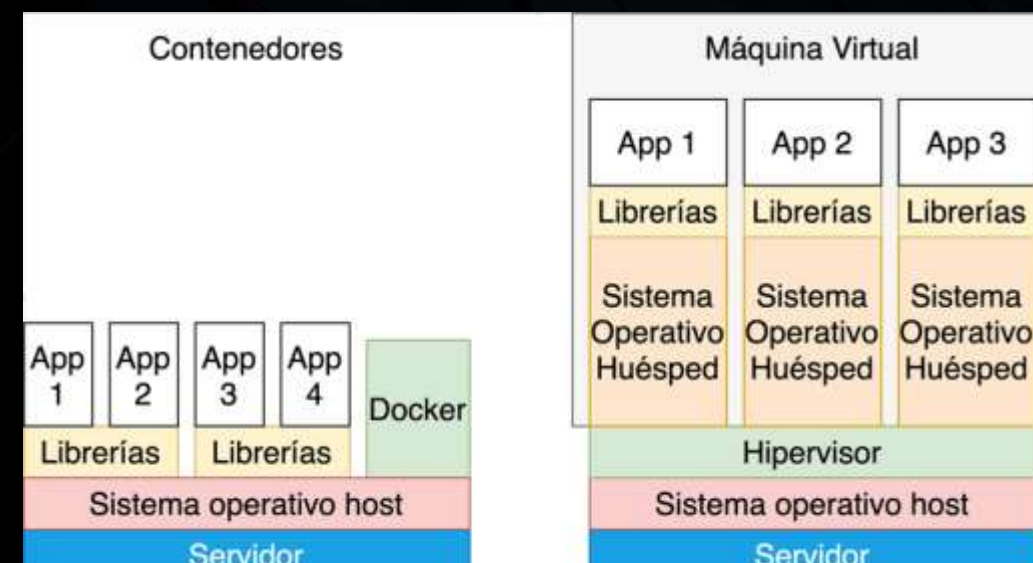
Un contenedor se diferencia de una máquina virtual porque no trae un sistema operativo completo, sino que comparte el kernel del host.

Esto hace que sea más ligero, rápido y portable.

Ejemplo de uso:

- Tienes una app en Java + MySQL → en lugar de instalar todo en tu máquina, lo empaquetas en contenedores:
 - `docker run mysql:latest`
 - `docker run miapp:1.0`

Así, la app siempre funciona igual en cualquier entorno (dev, test, producción).



CONTENEDORES - VENTAJAS

Tamaño muy reducido

- Ejemplo: una imagen base de Ubuntu puede pesar >70 MB, mientras que un microcontenedor como Alpine Linux pesa ~5 MB.

Seguridad mejorada

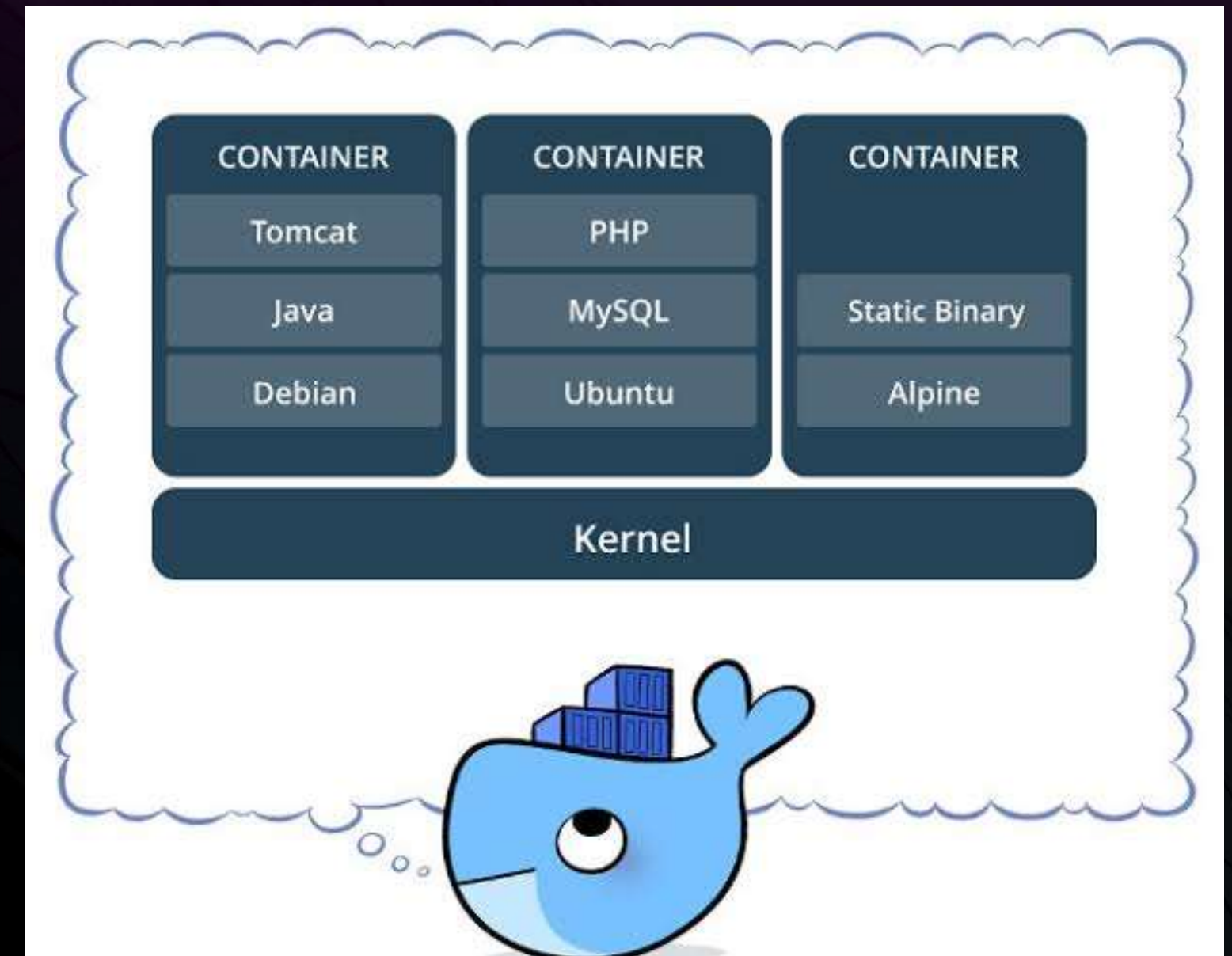
- Menos paquetes = menor superficie de ataque.
- Menos vulnerabilidades potenciales.

Arranque rápido

- Al tener menos componentes, los contenedores se levantan en segundos.

Eficiencia en despliegue

- Ideal para microservicios, donde necesitas cientos o miles de contenedores corriendo en paralelo.



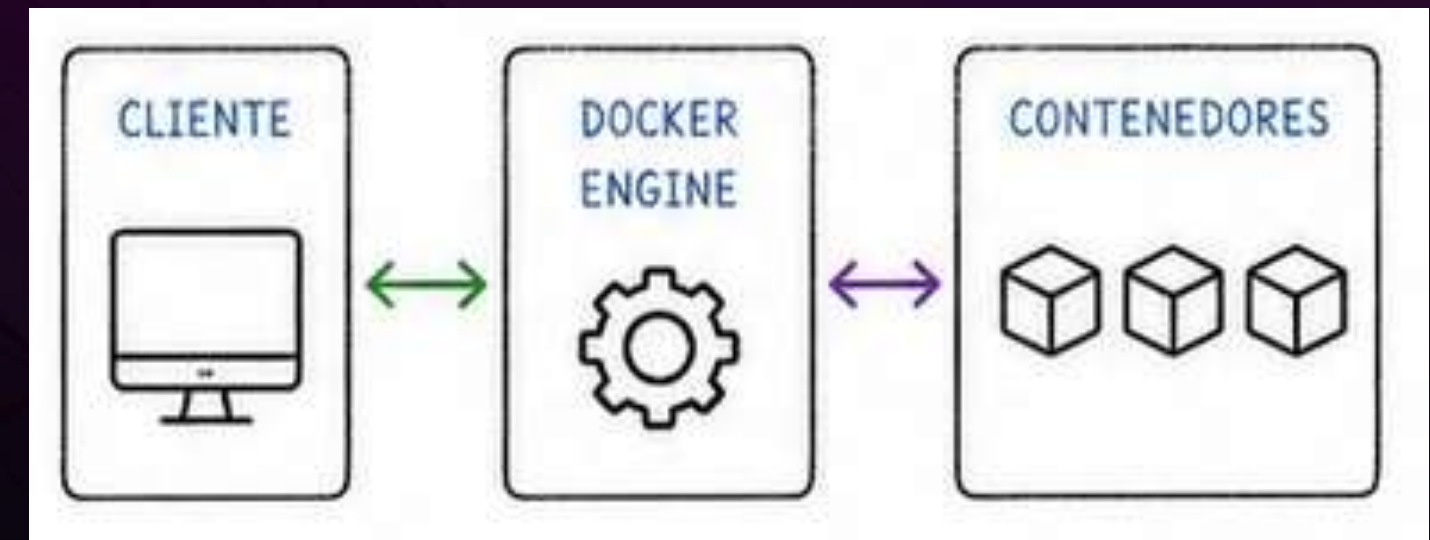
CONTENEDORES - DESVENTAJAS

- Menos herramientas disponibles
 - Al ser tan minimalistas, no tienen utilidades como bash, curl, apt-get, etc.
 - Puede complicar depurar errores dentro del contenedor.
- Curva de aprendizaje
 - Requiere más experiencia para construir imágenes bien optimizadas.
 - No es tan directo como usar imágenes grandes con todo incluido.
- Compatibilidad limitada
 - Algunas librerías o binarios esperan un sistema base más completo (ej. glibc vs musl en Alpine).
 - Puede haber problemas de compatibilidad en runtime.



DOCKER ENGINE

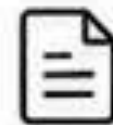
Es el motor que permite crear, ejecutar y administrar contenedores



DOCKER FILE

Es un archivo de texto con instrucciones para construir una imagen Docker

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "app.js"]
```



COMANDOS BASICOS

BUILD: Comando para construir la imagen

```
docker build -t  
mi-app:1.0 .
```

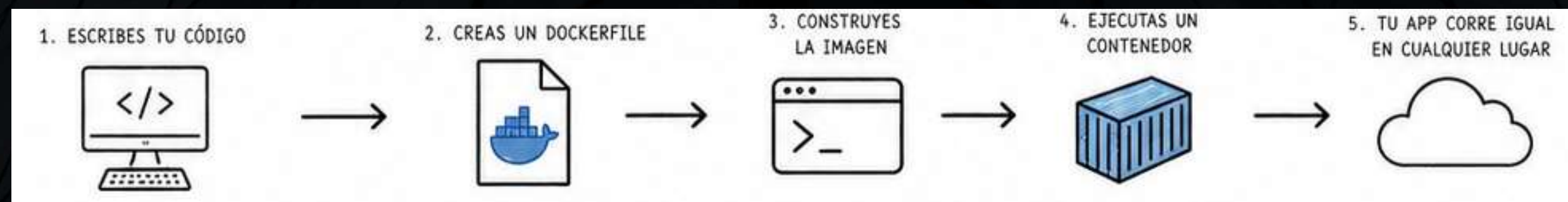


RUN: Comando para ejecutar el contenedor

```
docker run -d -p  
3000:3000 mi-app:1.0
```



FLUJO TIPICO EN DOCKER



COMO SERIA LA ANALOGIA DE MI RESTAURANTE

- Las instrucciones para armar la cocina en nuestra app es el Dockerfile
- El kit estandarizado de cocina, listo para reproducirse es la imagen Docker
- Una cocina funcionando a partir de ese kit es el contenedor
- La bodega donde guardamos versiones de los kits es el registro de imagenes
- Y abrir mas cocinas usando el mismo kit, es la ejecucion de mas instancias de la API



PORQUE ES IMPORTANTE DOCKERIZAR MI APLICACION

- Reduce el típico “en mi máquina funciona”, facilita desplegar versiones y permite reproducir el entorno de ejecución.
- Cada sede todavía necesita sus conexiones y secretos: por ejemplo, la dirección del proveedor o la llave de la bodega.



Y QUE ES ESO DE AMBIENTES ?

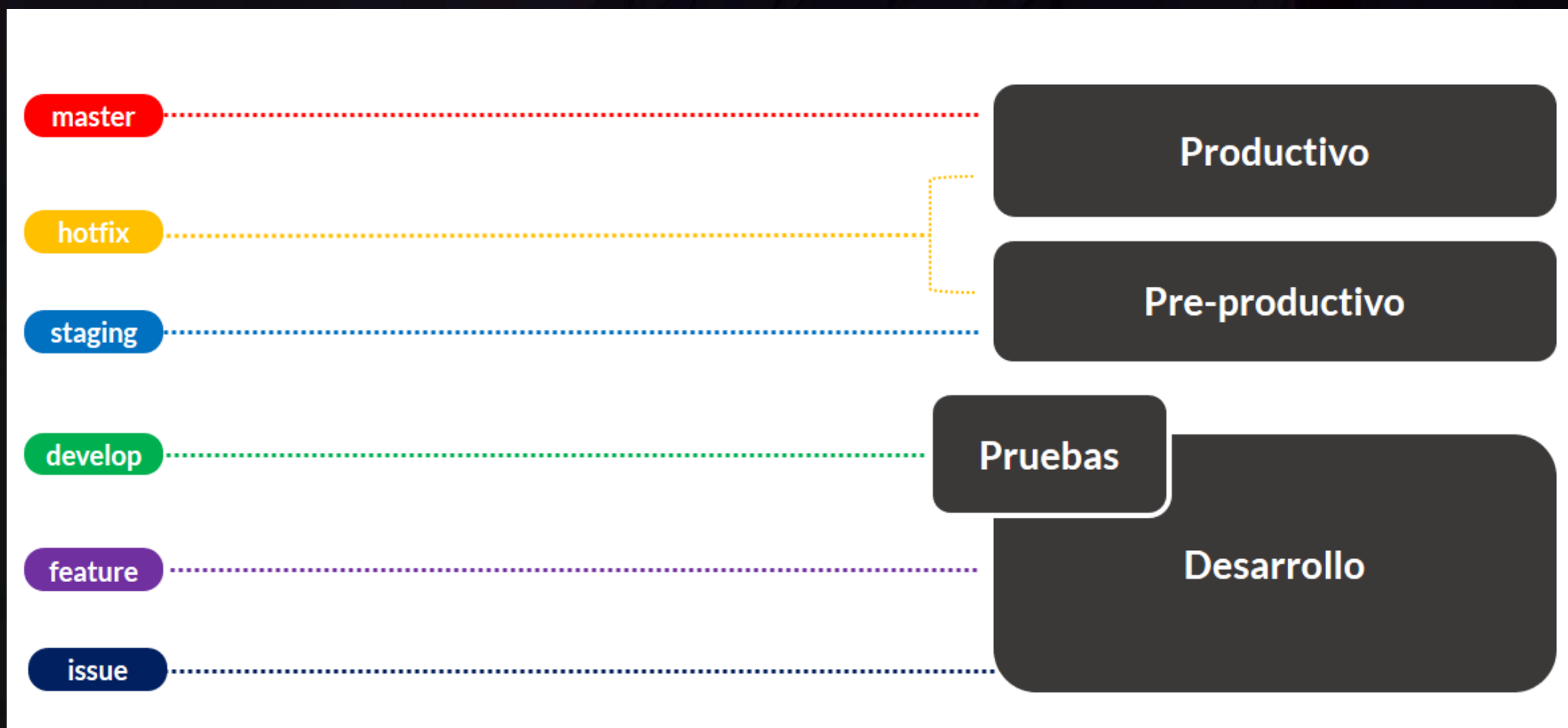


Un ambiente de pruebas existe para ganar confianza incremental antes de exponer cambios a usuarios.



DEV

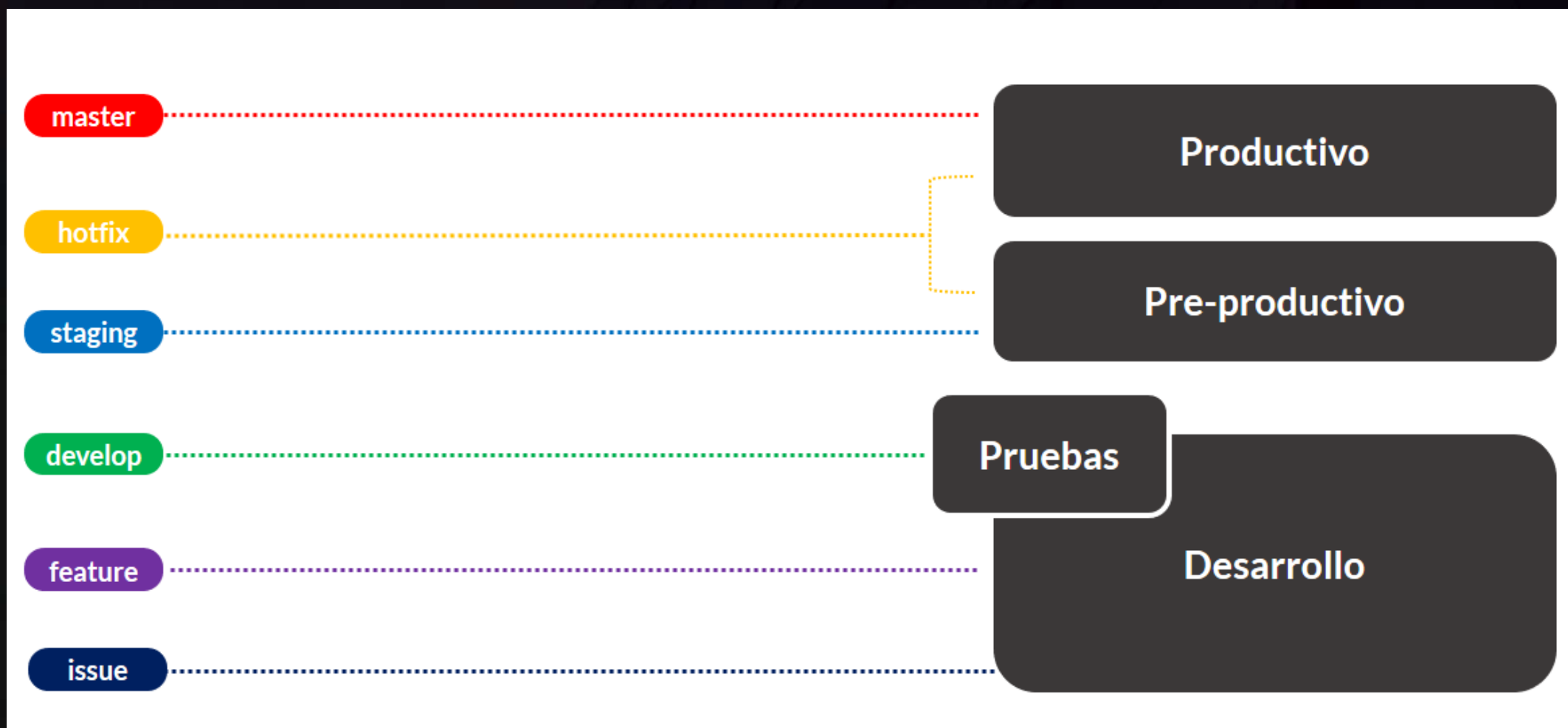
Este ambiente es local para los desarrolladores, donde cada uno puede trabajar de forma independiente, teniendo como punto de encuentro únicamente las ramas. Las reglas de este ambiente son:



- No se cuenta con ambiente de desarrollos compartidos para evitar errores en el código y conflictos con bases de datos.
- El ambiente debe manejar las mismas versiones de herramientas, lenguajes, frameworks y bases de datos que los ambientes productivos.

PRUEBAS - QA

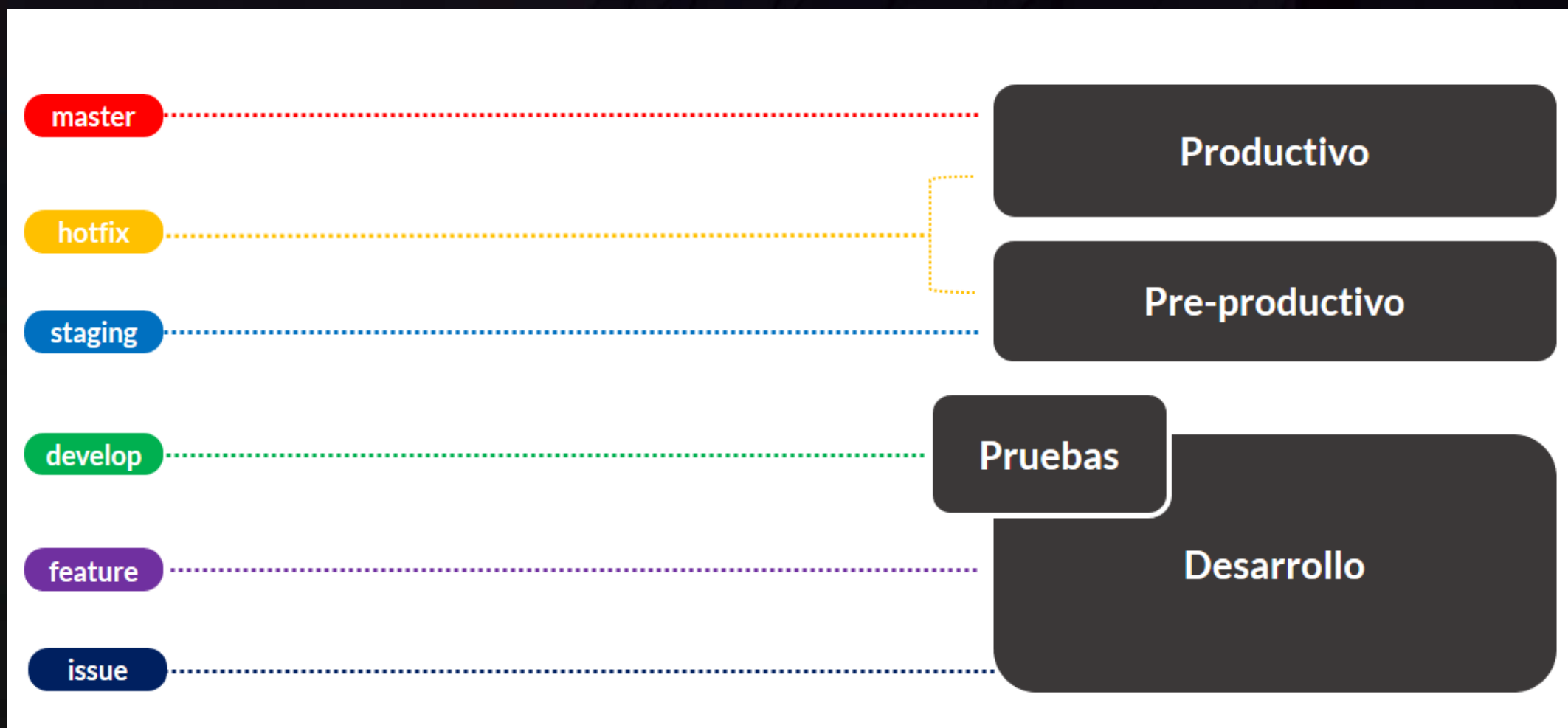
Este ambiente es homólogo a nivel de configuración del ambiente productivo. Está destinado para pruebas únicamente, las cuales son realizadas por el tester del proyecto. Las reglas de este ambiente son:



- El ambiente debe cumplir con una configuración idéntica al productivo, es decir a nivel de sistema operativo, servidor, permisos, roles, puertos, variables de ambiente, etc.
- El ambiente puede diferir en cuanto a algunas características de la arquitectura (replicación, autoescalamiento) pero no en sus componentes (servicios, cache, colas de tareas, etc).
- El equipo de pruebas es responsable de mantener este ambiente estable

PREPROD

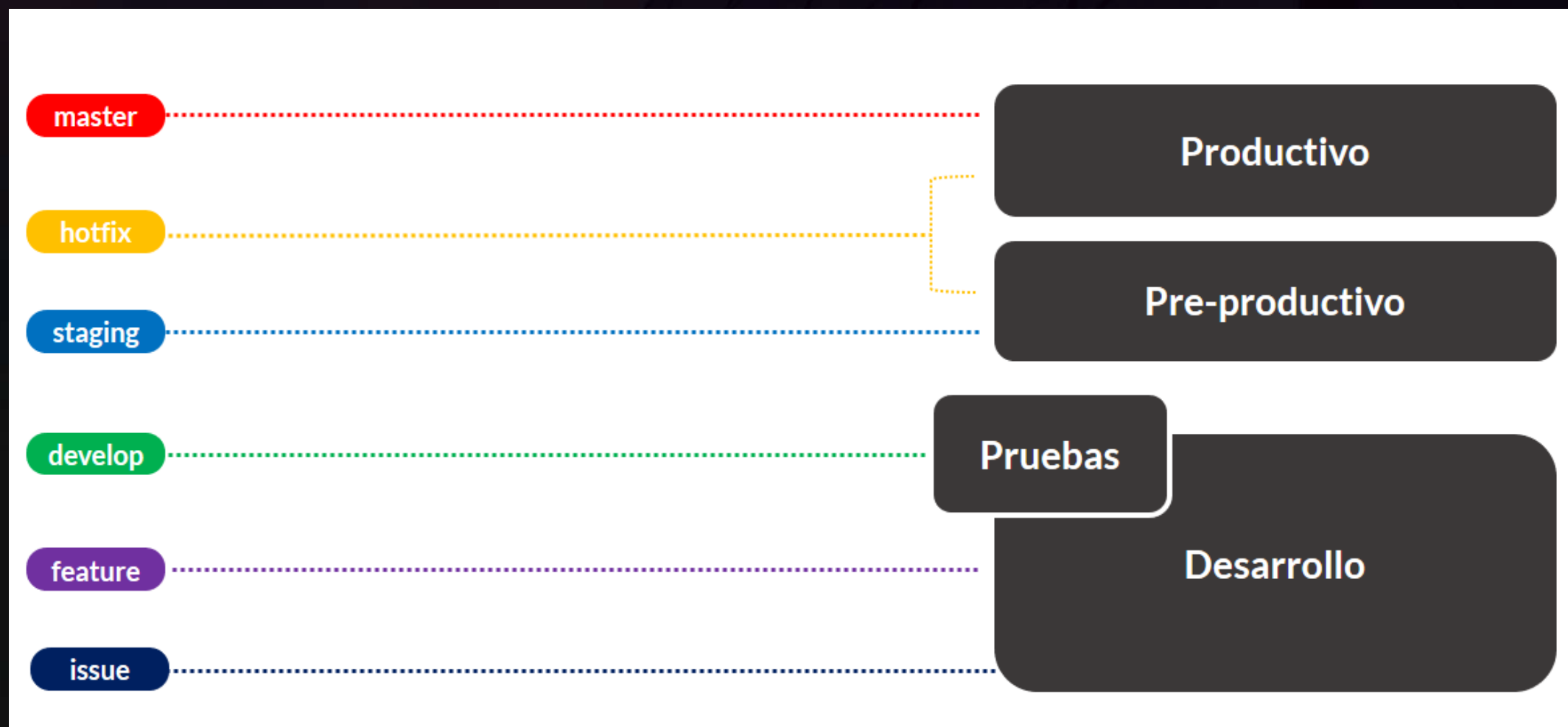
Este ambiente es homólogo a todo nivel con el ambiente productivo. Está destinado para realizar pruebas de aceptación con el líder funcional y el usuario final. Las reglas de este ambiente son:



- El ambiente debe cumplir con una configuración idéntica al productivo, es decir a nivel de sistema operativo, servidor, permisos, roles, puertos, variables de ambiente, etc.
- El ambiente no debe diferir en cuanto a arquitectura ni componentes frente al ambiente productivo.
- El equipo de operaciones es responsable de mantener este ambiente estable

PRODUCCION

Este ambiente es el de producción de la aplicación. Las reglas de este ambiente son:



- Siempre debe tener una versión estable de la aplicación en ejecución
- El equipo de operaciones es responsable de mantener este ambiente estable
- El equipo de operaciones son los únicos que pueden desplegar una versión anterior de la aplicación si se considera necesario

Y COMO COORDINAMOS ESA REPRODUCCION DE NUESTRAS COCINAS?



El gerente de operaciones de la franquicia: coordina las cocinas que Docker permite reproducir, para que el restaurante siga atendiendo aunque aumenten los pedidos o falle alguna cocina

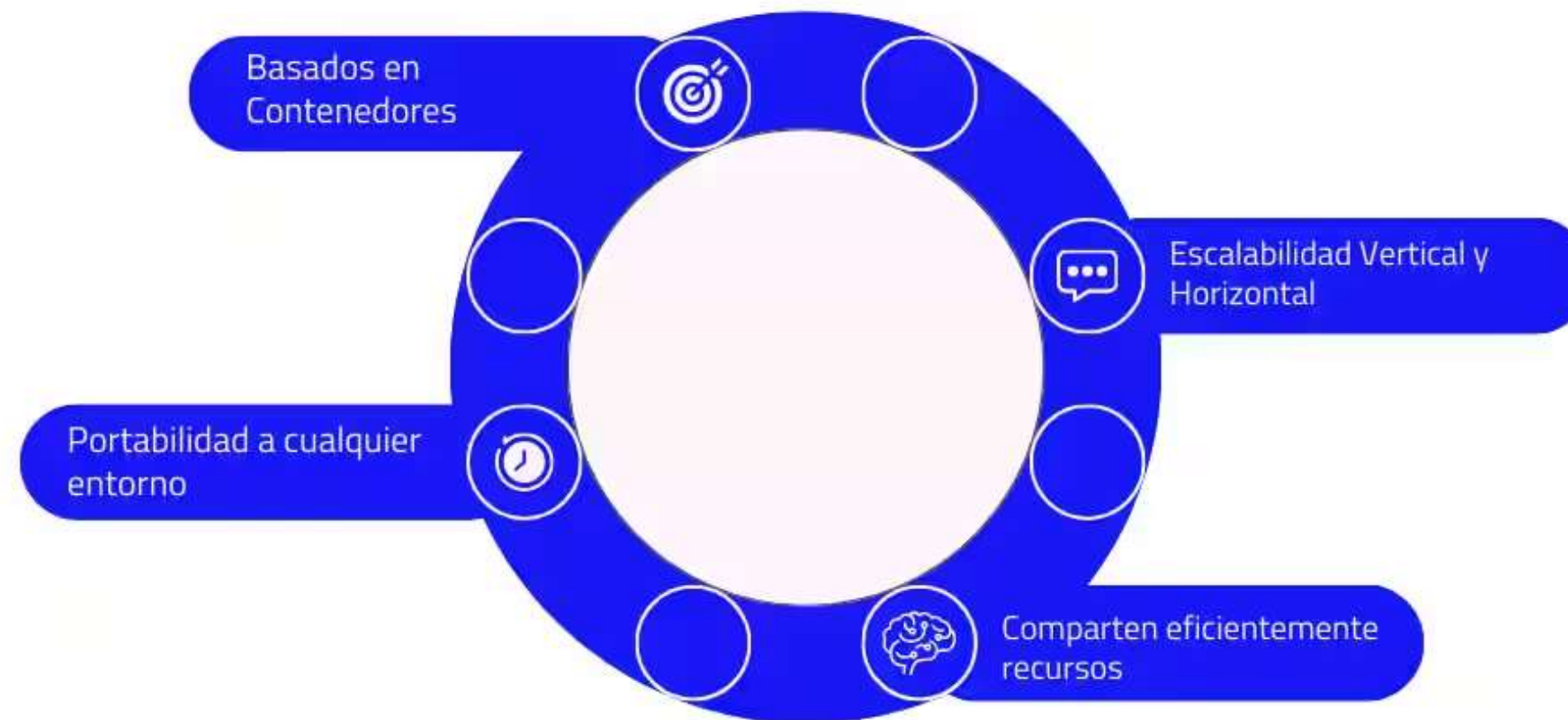


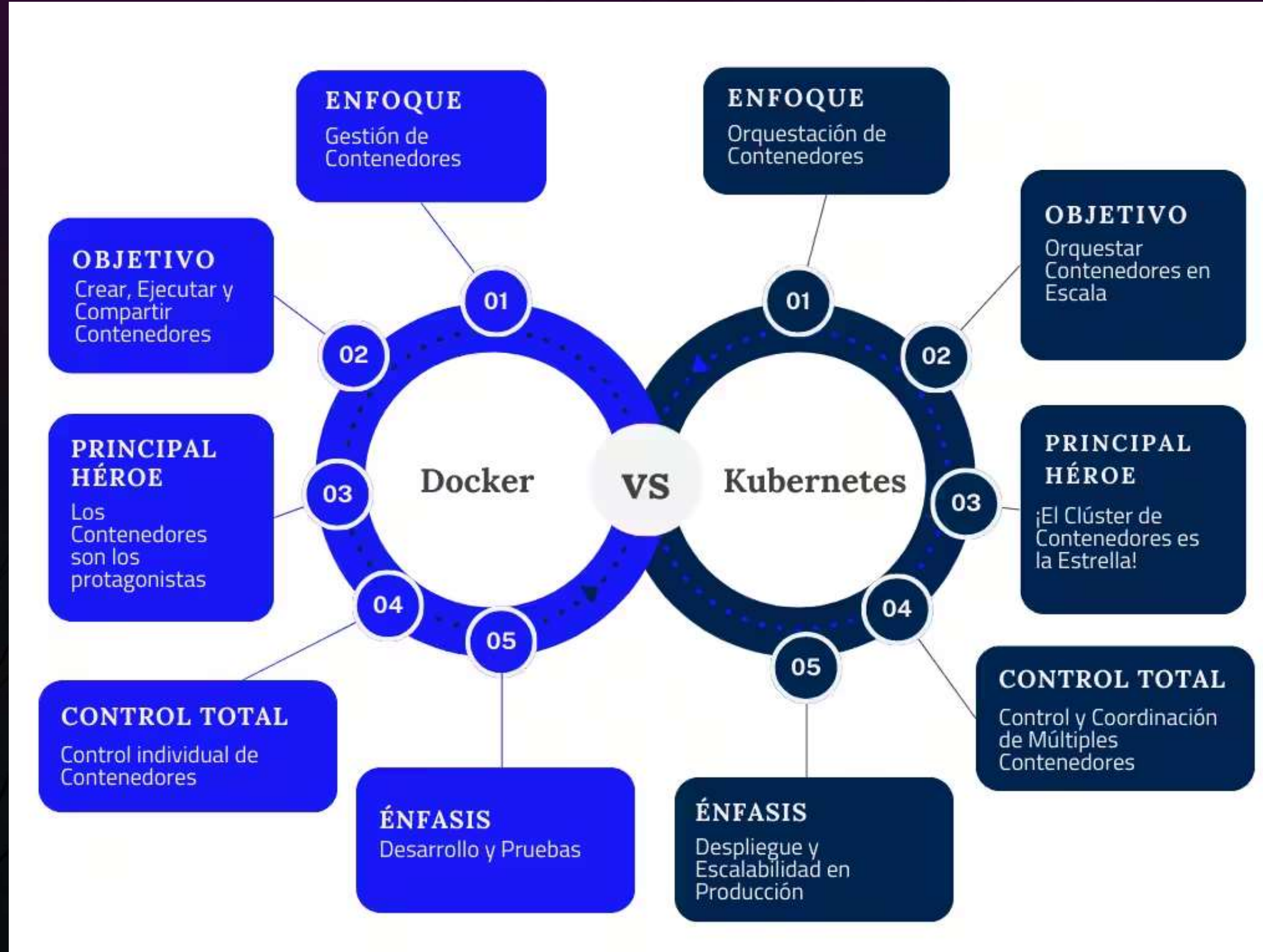
kubernetes

Kubernetes (K8s) es un orquestador de contenedores.

- Imagina que tienes decenas o cientos de contenedores Docker. Administrarlos uno por uno sería un caos.
- Kubernetes se encarga de:
 - Programar y desplegar contenedores en clústeres de máquinas.
 - Escalar automáticamente (más o menos contenedores según la carga).
 - Balancear carga entre contenedores.
 - Recuperar fallos (si un contenedor muere, lo reinicia).
 - Actualizar sin downtime (rolling updates, blue/green deployments).

DOCKER & KUBERNETES





DOCKER CREA Y EJECUTA CONTENEDORES KUBERNETES LOS COORDINA EN GRAN ESCALA.

Docker = Contenedor individual → como una “caja” que guarda una aplicación lista para usar.

Kubernetes = Puerto marítimo → organiza, coordina y distribuye miles de contenedores en barcos, asegurándose de que todo funcione sin problemas.





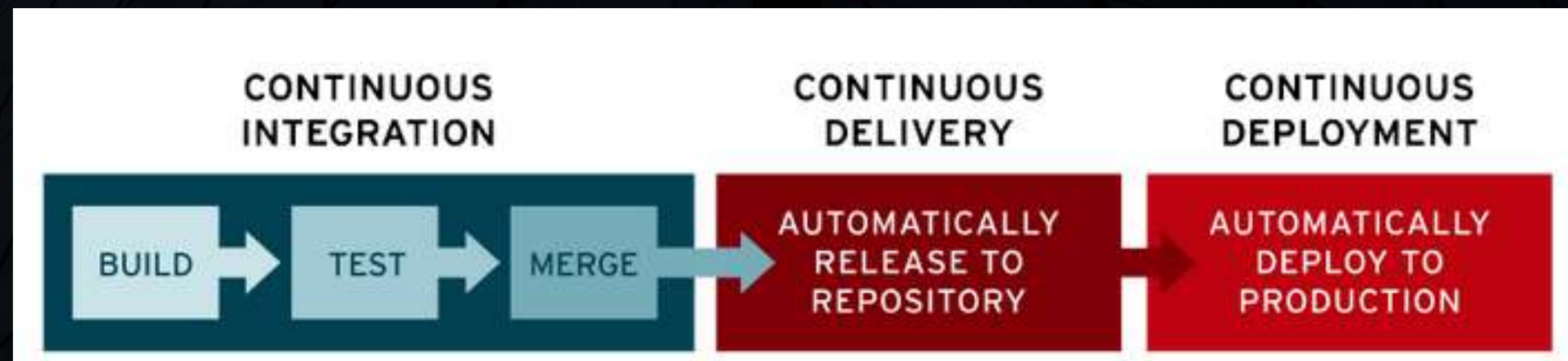
Ahora imagina que un chef modifica la receta de la hamburguesa. No queremos que cada sede reciba una nota por WhatsApp y haga el cambio como pueda. Queremos un proceso controlado que revise la receta, la pruebe y prepare su distribución.

**NECESITAMOS
AUTOMATIZAR**



CI/CD

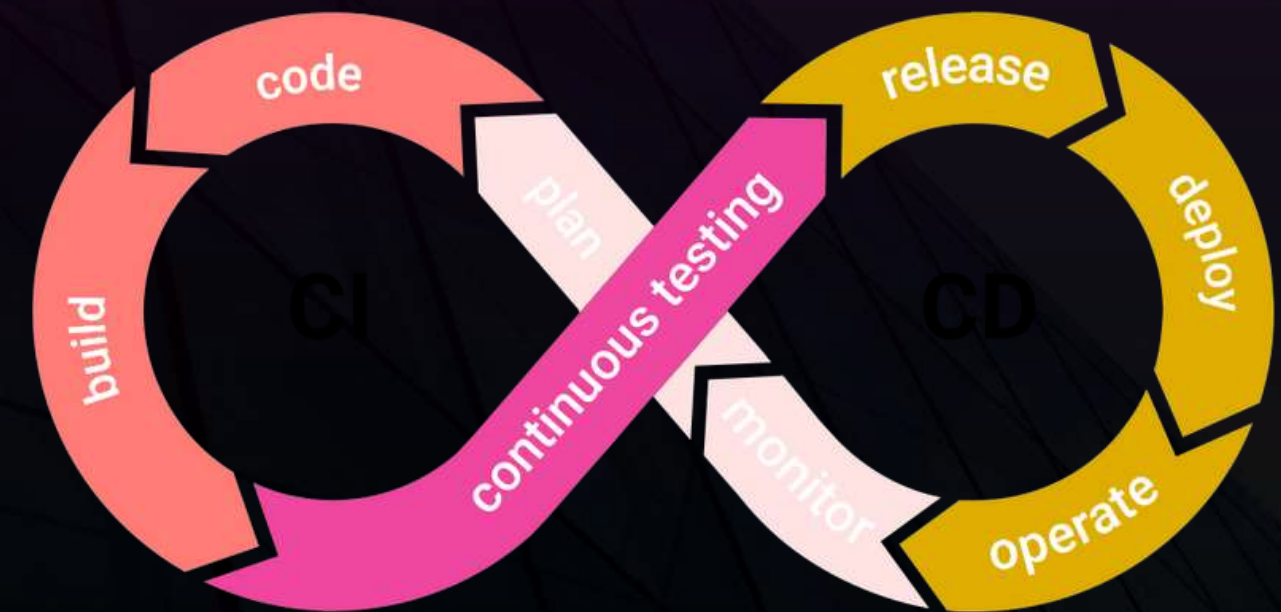
Integración y despliegue continuo – describen una forma de trabajo donde los cambios de código pasan por una cadena automatizada que construye, prueba, empaqueta y despliega software de manera confiable y repetible





CI/CD

- Los desarrolladores suben su código con frecuencia a un repositorio central (ej. GitHub, GitLab, Bitbucket).
- Cuando trabajamos en local seguimos una secuencia logica para garantizar que nuestro codigo funcione, compilamos, ejecutamos pruebas y corremos nuestra aplicacion en el puerto que le damos.... pero todo ese proceso manual es tedioso
- Con CI/CD Cada vez que hay un cambio, un pipeline ejecuta:
 - Compilación del código
 - Ejecución de pruebas unitarias
 - Análisis de calidad (ej. SonarQube, linters)
 - Detecta errores rápidamente antes de que lleguen a producción.





CI/CD

En Integración Continua (CI), cada commit o Pull Request se integra temprano para detectar conflictos y defectos cuando todavía son baratos de corregir.

La Entrega/Despliegue Continua (CD) extiende ese flujo hacia ambientes reales, de modo que el software pueda llegar a usuarios con menor riesgo, gracias a lotes pequeños y pasos verificables.

- Continuous Delivery: asegura que la aplicación esté siempre lista para ser desplegada. El despliegue puede ser manual.
- Continuous Deployment: los cambios pasan automáticamente a producción si superan todas las pruebas.

COMO LO MEDIMOS?

Para poder medir que tan bien se está llevando el proceso de CI/CD y su impacto de valor, existen métricas estandar que nos ayudan,

Una de las maneras de hacerlo es con dora, que tiene un conjunto de indicadores clave utilizados para medir el rendimiento de los equipos de desarrollo de software. Estas métricas, definidas por el grupo DevOps Research and Assessment (DORA), incluyen:



**DEPLOYMENT
FREQUENCY**



**LEAD TIME
FOR CHANGES**



**CHANGE
FAILURE RATE**



**TIME TO RESTORE
SERVICE**



CON QUE PODEMOS
AUTOMATIZAR... CON
PIPELINES

AUTOMATIZACION CON PIPELINES

Un pipeline es una serie de etapas automatizadas que transforman el código fuente en una aplicación lista para usarse.

1. Build (Compilación):

- a. Instalar dependencias, compilar código, empaquetar (ej. npm install, mvn package, docker build).

2. Test (Pruebas):

- a. Unitarias, integración, contract testing, e2e. – Automatizadas para validar cada commit.

3. Analysis (Análisis de calidad y seguridad)

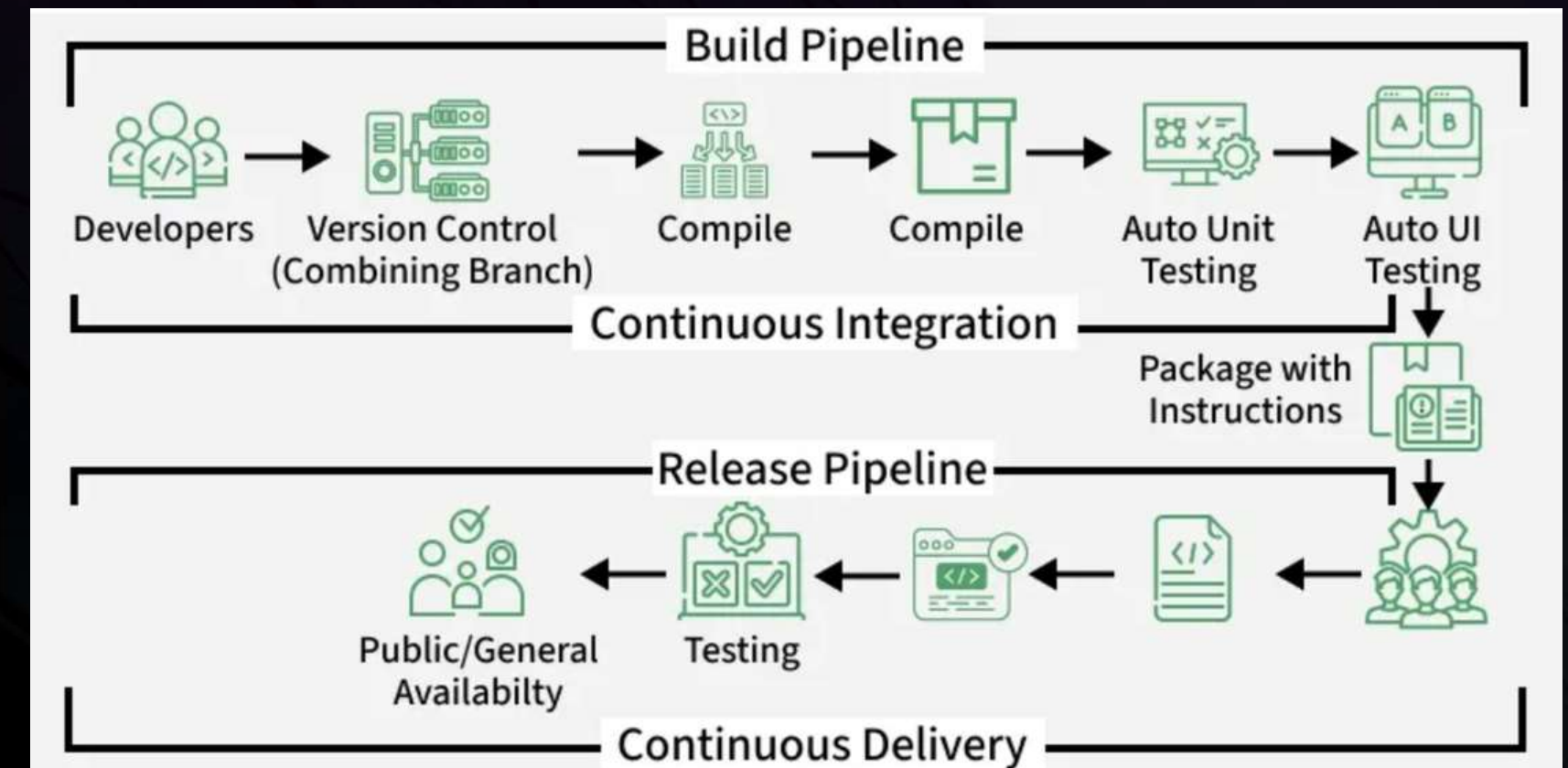
- a. Estático (linters, SonarQube, ESLint).
- b. Seguridad (Snyk, OWASP Dependency Check).

4. Deploy (Despliegue)

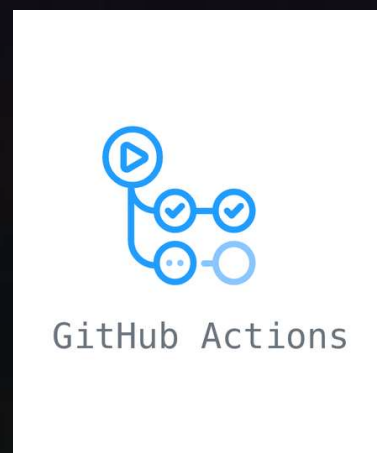
- a. Despliegue a entornos: DEV, QA, Sandbox, Preprod y producción.
- b. Uso de contenedores (Docker, Kubernetes).
- c. Estrategias: blue/green, canary, rolling updates.

5. Monitoring (Monitoreo y Feedback) – OPCIONAL

- a. Integración con herramientas de logging y monitoreo (Prometheus, Grafana, ELK, Datadog).
- b. Alertas automáticas si algo falla.



HERRAMIENTAS POPULARES PARA PIPELINES



GitHub
Actions



GitLab
CI/CD



Jenkins

Jenkins



CircleC
I

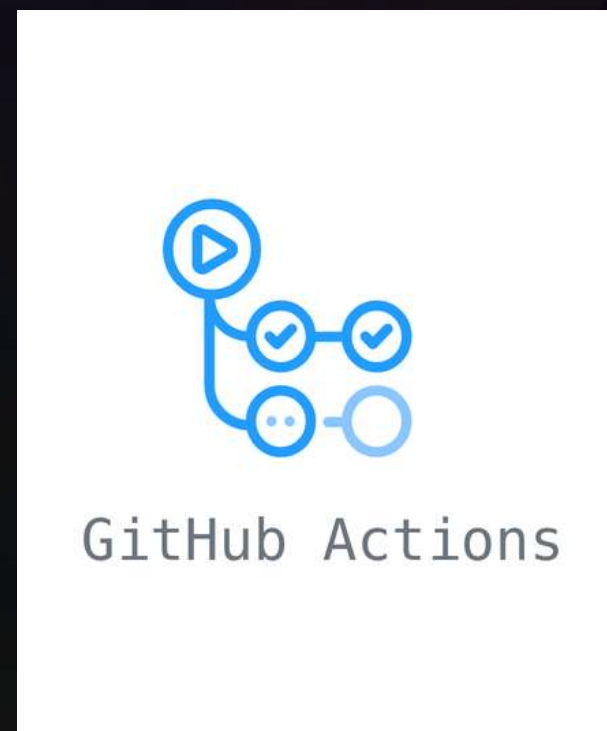


Azure DevOps
Pipelines



GITHUB ACTIONS

Es la herramienta de CI/CD (Integración y Despliegue Continuo) integrada en GitHub que permite automatizar flujos de trabajo directamente desde tu repositorio.



Es un servicio que corre workflows (flujos de trabajo) definidos en archivos YAML.

Estos workflows se ejecutan automáticamente en respuesta a eventos dentro del repositorio (ejemplo: un push, un pull request, creación de release).

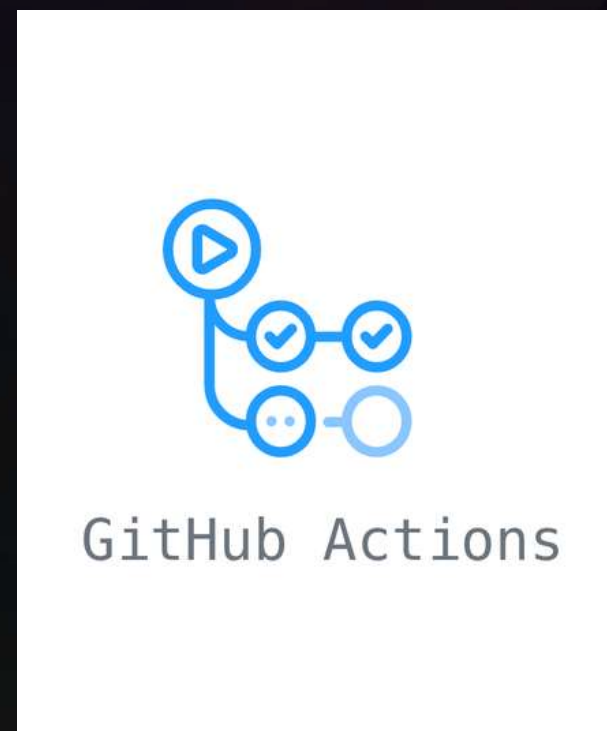
Se usan para tareas como:

- Compilar y probar código.
- Hacer despliegues automáticos.
- Ejecutar análisis de calidad.
- Publicar paquetes (npm, Maven, Docker Hub, etc.).

GITHUB ACTIONS

los componentes principales son:

- **WORKFLOW:** El archivo .yml en el directorio .github/workflows, donde define el pipeline con todos sus pasos
- **EVENTS:** Es lo que dispara el workflow – push cuando suben cambios, pull_request cuando abren un PR, schedule tareas programadas
- **JOBS:** Cada workflow tiene uno o varios jobs – un Job es un conjunto de pasos que corren en un mismo ambiente (Windows, Ubuntu, Mac)
- **STEPS:** Acciones o comandos que se ejecutan dentro del JOB – instalar dependencias, correr test, etc
- **ACTIONS:** Son scripts predefinidos que se pueden reutilizar





GITHUB ACTIONS

```
name: Java CI with Maven

on:
  push:
    branches: [ "main" ]    # Se ejecuta al hacer push en main
  pull_request:
    branches: [ "main" ]    # También en los PR hacia main

jobs:
  build:
    runs-on: ubuntu-latest  # Sistema operativo del runner

    steps:
      - name: Checkout repo
        uses: actions/checkout@v3    # Clona el repositorio

      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          distribution: 'temurin'    # OpenJDK (Eclipse Temurin)
          java-version: '17'

      - name: Build with Maven
        run: mvn -B package --file pom.xml
        # -B = modo batch (sin preguntas interactivas)

      - name: Run Unit Tests
        run: mvn test
```

```
deploy:
  runs-on: ubuntu-latest
  needs: build    # Solo se ejecuta si el build pasó
  steps:
    - name: Checkout repo
      uses: actions/checkout@v3

    - name: Set up JDK 17
      uses: actions/setup-java@v3
      with:
        distribution: 'temurin'
        java-version: '17'

    - name: Build and Publish to GitHub Packages
      run: mvn deploy -s $GITHUB_WORKSPACE/.m2/settings.xml
      env:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```



YA CONOCIMOS COMO VA A FUNCIONAR LA SEGURIDAD EN NUESTRA COCINA PERO....

En el restaurante, las credenciales son las llaves de la bodega, la clave de la caja fuerte y los accesos a los proveedores.

No las escribimos en el recetario ni las pegamos en la cocina: las guardamos por separado y las entregamos únicamente a quien las necesita.





PARA ESO UTILIZAMOS LOS SECRETS

Para usar secretos en GitHub Actions, debes almacenar la información sensible de forma segura en la configuración de tu repositorio y llamarla en tu archivo YAML usando la sintaxis `${{ secrets.NOMBRE_DEL_SECRETO }}`



Por ejemplo, el pipeline necesita una credencial para publicar la imagen Docker. La guardas en:
Settings → Secrets and variables → Actions → New repository secret



ALGUNAS REGLAS

Las llaves no van en el código expuesto es decir no debes subir contraseñas, archivos .env reales ni secretos a Git.

Las llaves no van dentro del kit, es decir no incorpores credenciales a la imagen Docker.

Cada sede tiene sus llaves: separar credenciales de desarrollo, pruebas y producción.

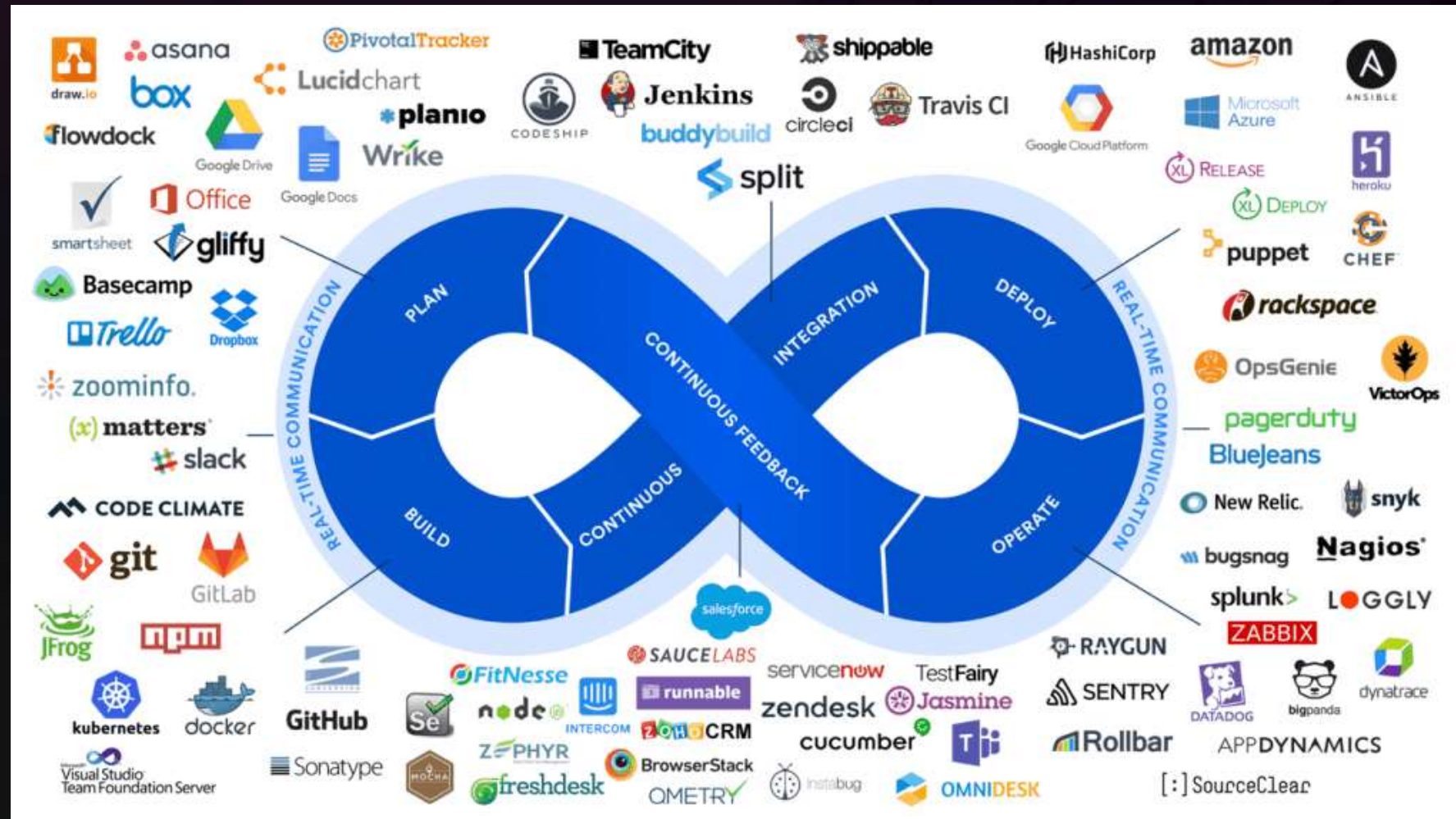
No anunciar las claves en voz alta: evitar imprimir secretos en logs.

Si una llave se filtra, se cambia: borrar el archivo del repositorio no invalida la credencial.



DEVOPS

DEVOPS

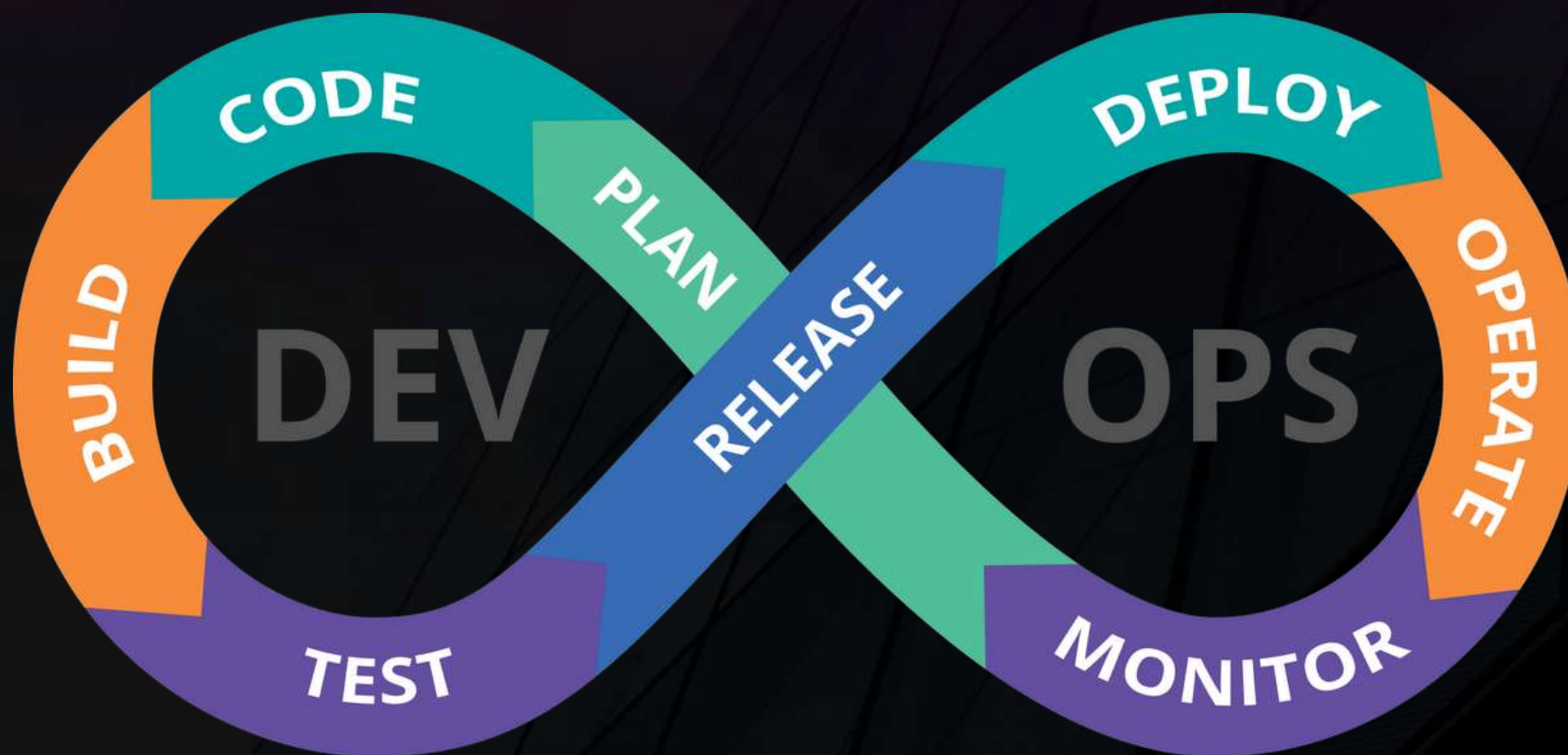


DevOps es una práctica que combina personas, procesos y tecnología para acortar el ciclo de vida del desarrollo de software, integrando los equipos de desarrollo (Dev) y operaciones (Ops) con el objetivo de entregar valor de manera continua, rápida y confiable a los clientes. Se enfoca en la colaboración, la automatización de tareas repetitivas y el uso de ciclos de retroalimentación constante para mejorar la calidad del producto y la eficiencia del equipo.

DEVOPS

¿Qué es DevOps?

- Una cultura
- Un camino para administrar el desarrollo de aplicaciones que generen valor.
- Un esquema organizacional.





DEVOPS

N
Dev~~Ops~~

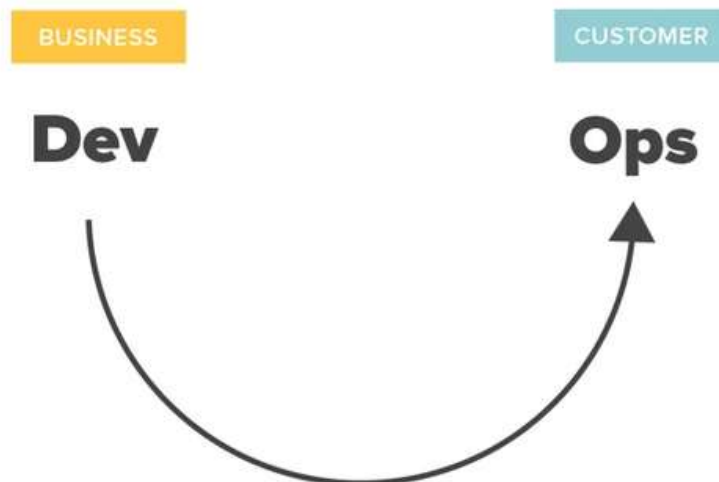
¿Qué no es DevOps?

- Un producto que se pueda instalar
- Un servicio que se pueda comprar
- Un departamento que se pueda crear

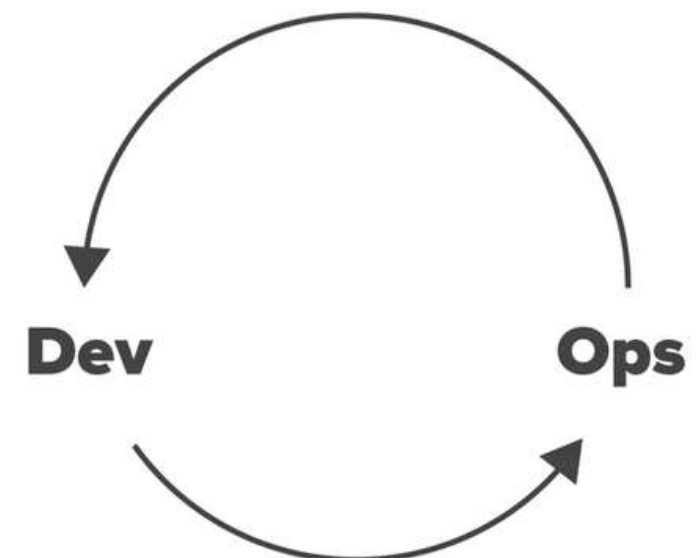
DEVOPS

Las tres vias de adopcion de DEVOPS

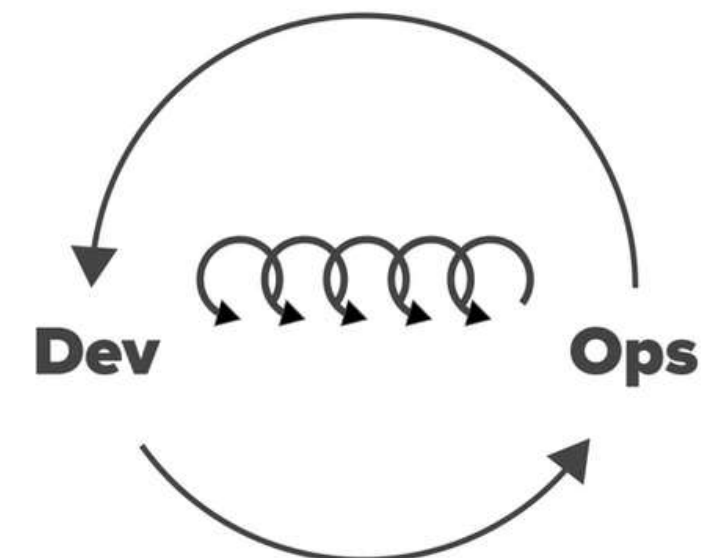
1 The First Way: Systems Thinking



2 The Second Way: Amplify Feedback Loops



3 The Third Way: Culture of Continual Experimentation And Learning





CALMS

CALMS es un marco para evaluar/adoptar DevOps que integra Cultura, Automatización, enfoque Lean, Medición y Sharing (compartir): personas primero, flujos automatizados y ligeros, decisiones basadas en métricas y aprendizaje abierto.

Valores CALMS



Lean busca maximizar el valor entregado al cliente y minimizar los desperdicios.



IT AS A SERVICE

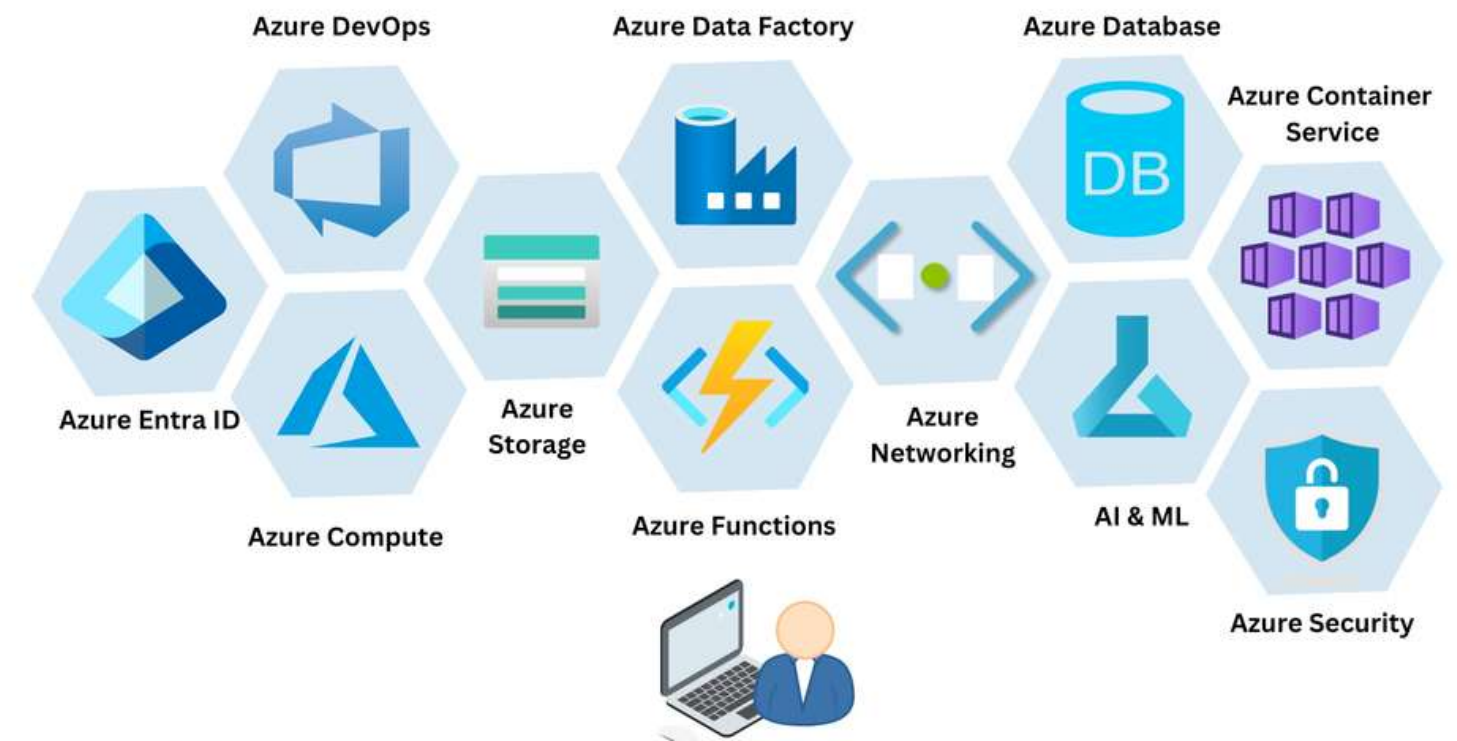
IT AS A SERVICE

Convierte a TI en un proveedor interno de servicios con un catálogo claro, acuerdos de servicio y costos visibles. En lugar de tickets ambiguos,

los equipos consumen productos de plataforma: crear un entorno de aplicación, solicitar una base de datos administrada, registrar un dominio, activar observabilidad, gestionar identidades. Cada ítem del catálogo define qué entrega, cómo se solicita, cuánto tarda, cuánto cuesta y qué responsabilidades mantienen cliente y proveedor (modelo RACI).

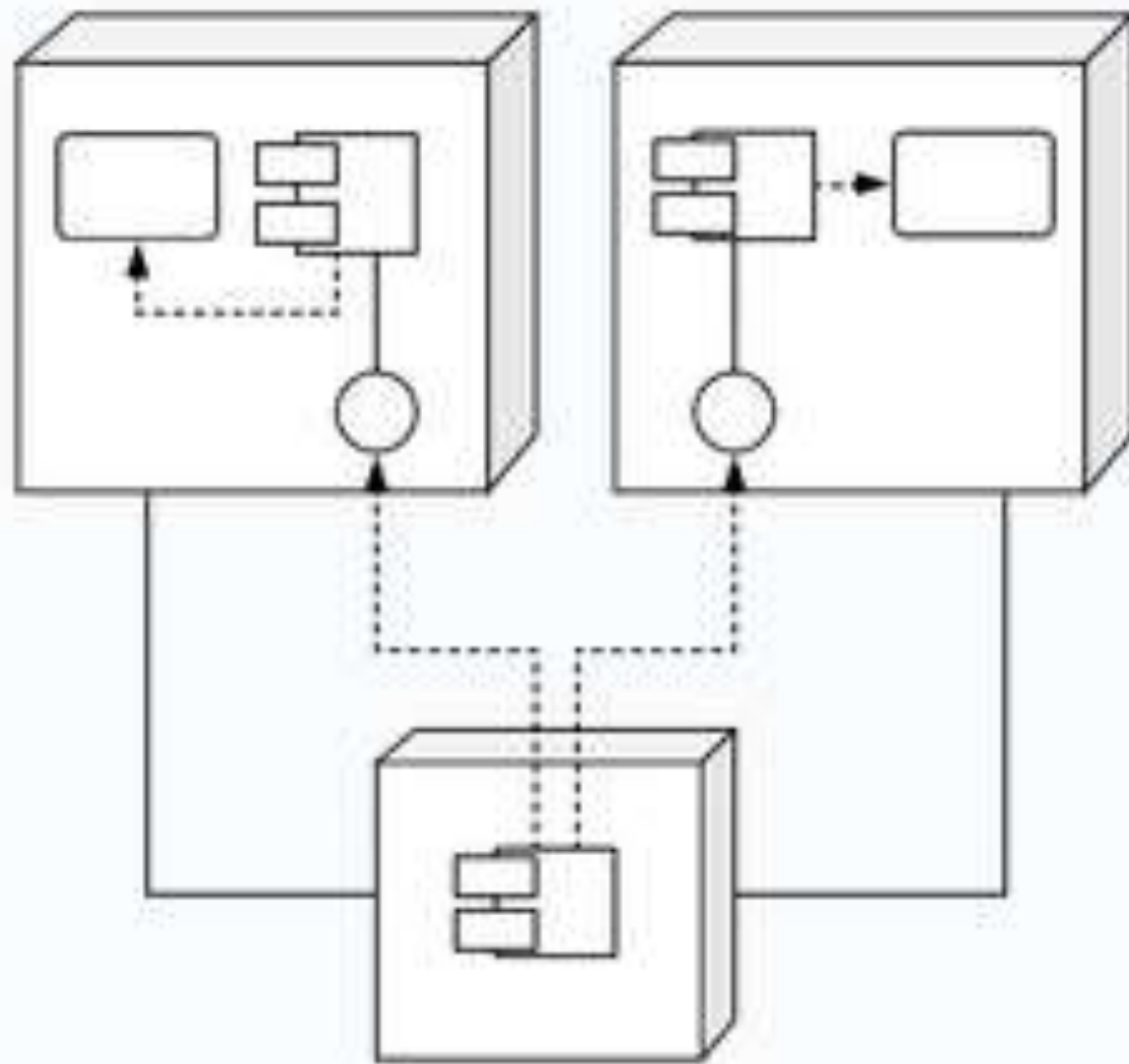


IT AS A SERVICE





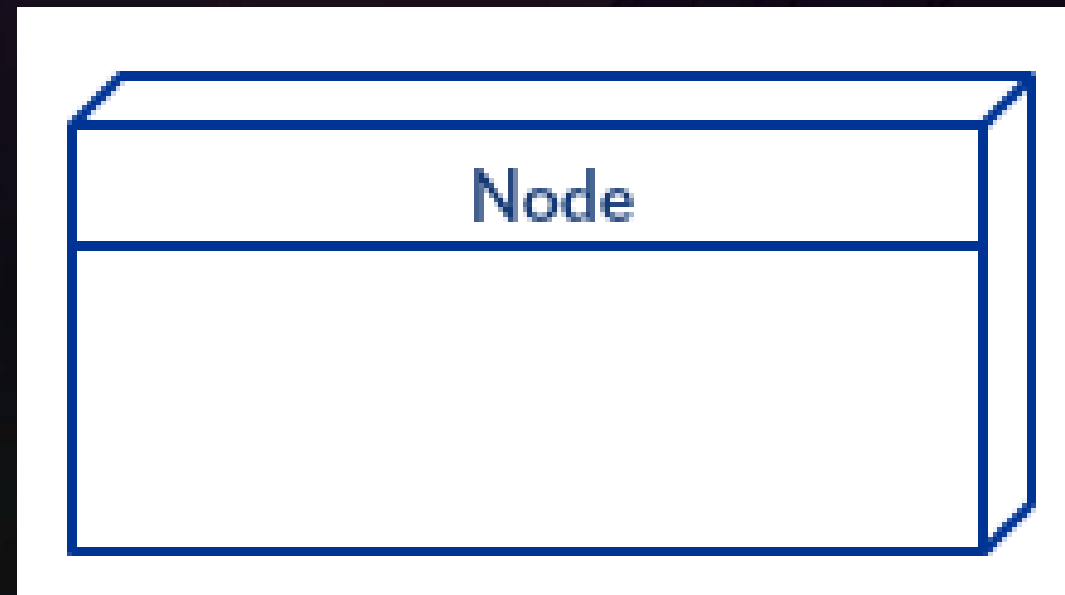
MODULO DE DIAGRAMACION #6: DIAGRAMA DE DESPLIEGUE



Los diagramas de despliegue se utilizan para visualizar los procesadores/ nodos/dispositivos de hardware de un sistema, los enlaces de comunicación entre ellos y la colocación de los archivos de software en ese hardware.

Un diagrama de despliegue es un tipo de diagrama UML que muestra la arquitectura de ejecución de un sistema, incluyendo nodos como entornos de ejecución de hardware o software, y el middleware que los conecta.

NOTACIONES



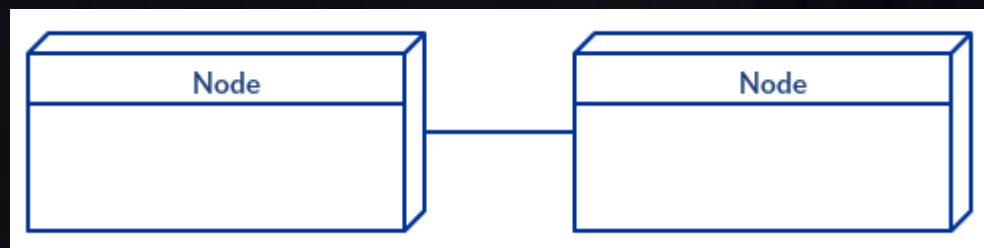
Un nodo, representado como un cubo, es una entidad física que ejecuta uno o más componentes, subsistemas o ejecutables. Un nodo podría ser un elemento de hardware o software.



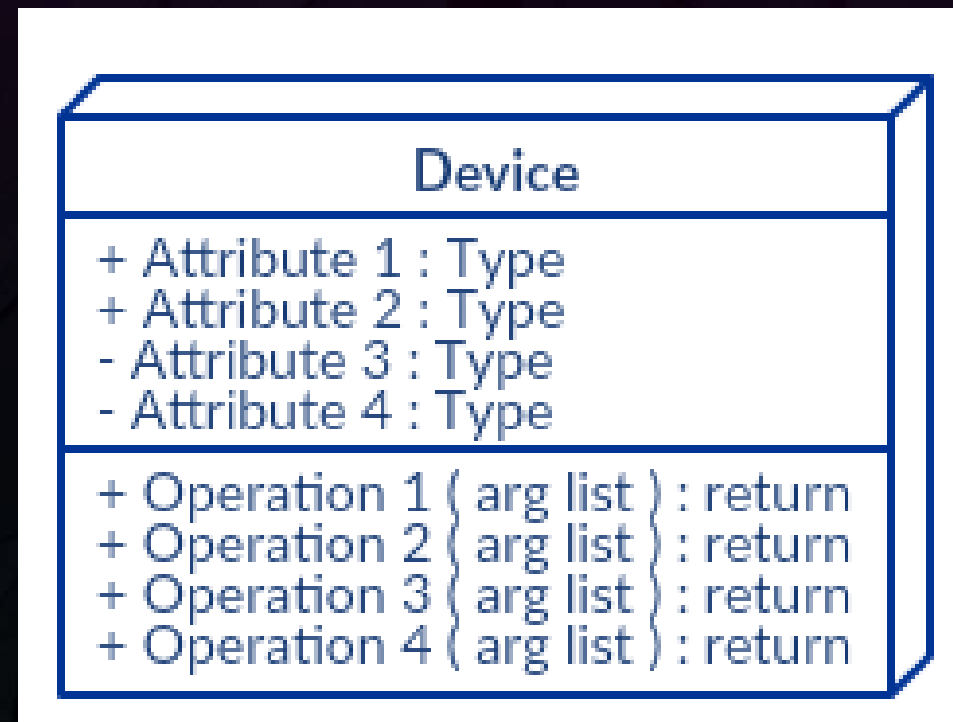
Artefactos del diagrama de despliegue Los artefactos son elementos concretos que son causados por un proceso de desarrollo.

Ejemplos de artefactos son las bibliotecas, archivos, archivos de configuración, archivos ejecutables, etc.

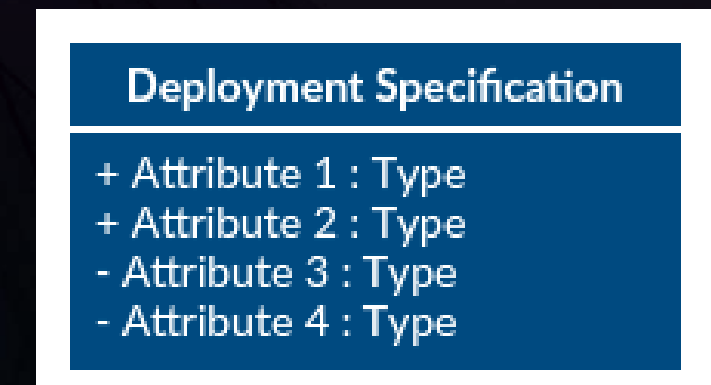
NOTACIONES



Esto está representado por una línea sólida entre dos nodos.
Muestra el camino de la comunicación entre los nodos.



Un dispositivo es un nodo que se utiliza para representar un recurso físico computacional en un sistema. Un ejemplo de un dispositivo es un servidor de aplicaciones.



Especificación despliegue Las especificaciones de despliegue son un archivo de configuración, como un archivo de texto o un documento XML. Describe cómo se despliega un artefacto en un nodo.



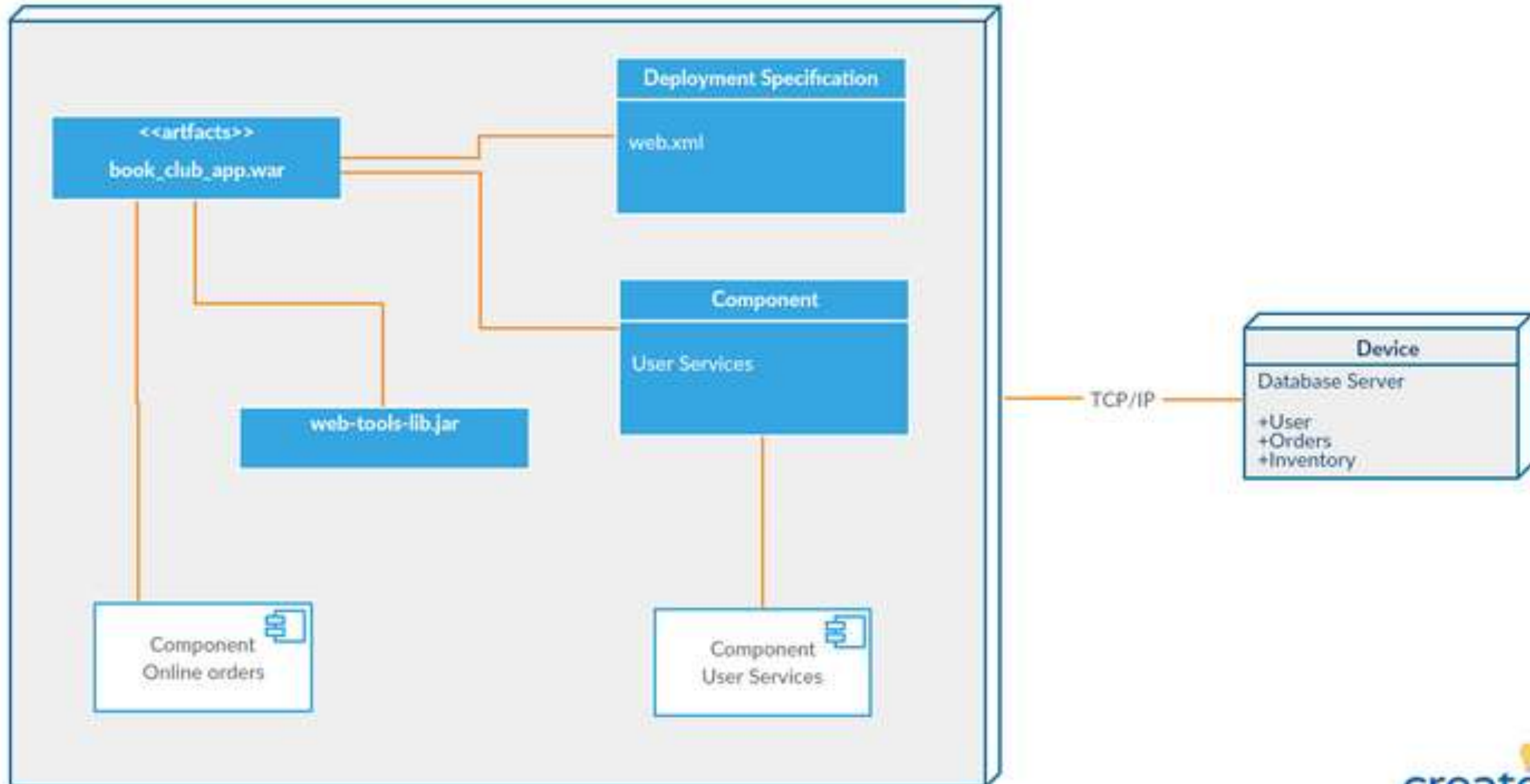
COMO DIBUJARLO?

- Paso 1: Identifique el propósito de su diagrama de despliegue. Y para hacerlo, necesitas identificar los nodos y dispositivos dentro del sistema que visualizarás con el diagrama.
- Paso 2: Averiguar las relaciones entre los nodos y los dispositivos. Una vez que sepas cómo están conectados, procede a añadir las asociaciones de comunicación al diagrama.
- Paso 3: Identificar qué otros elementos, como los componentes, objetos activos, necesitan ser añadidos para completar el diagrama.
- Paso 4: Añada las dependencias entre los componentes y los objetos según sea necesario.

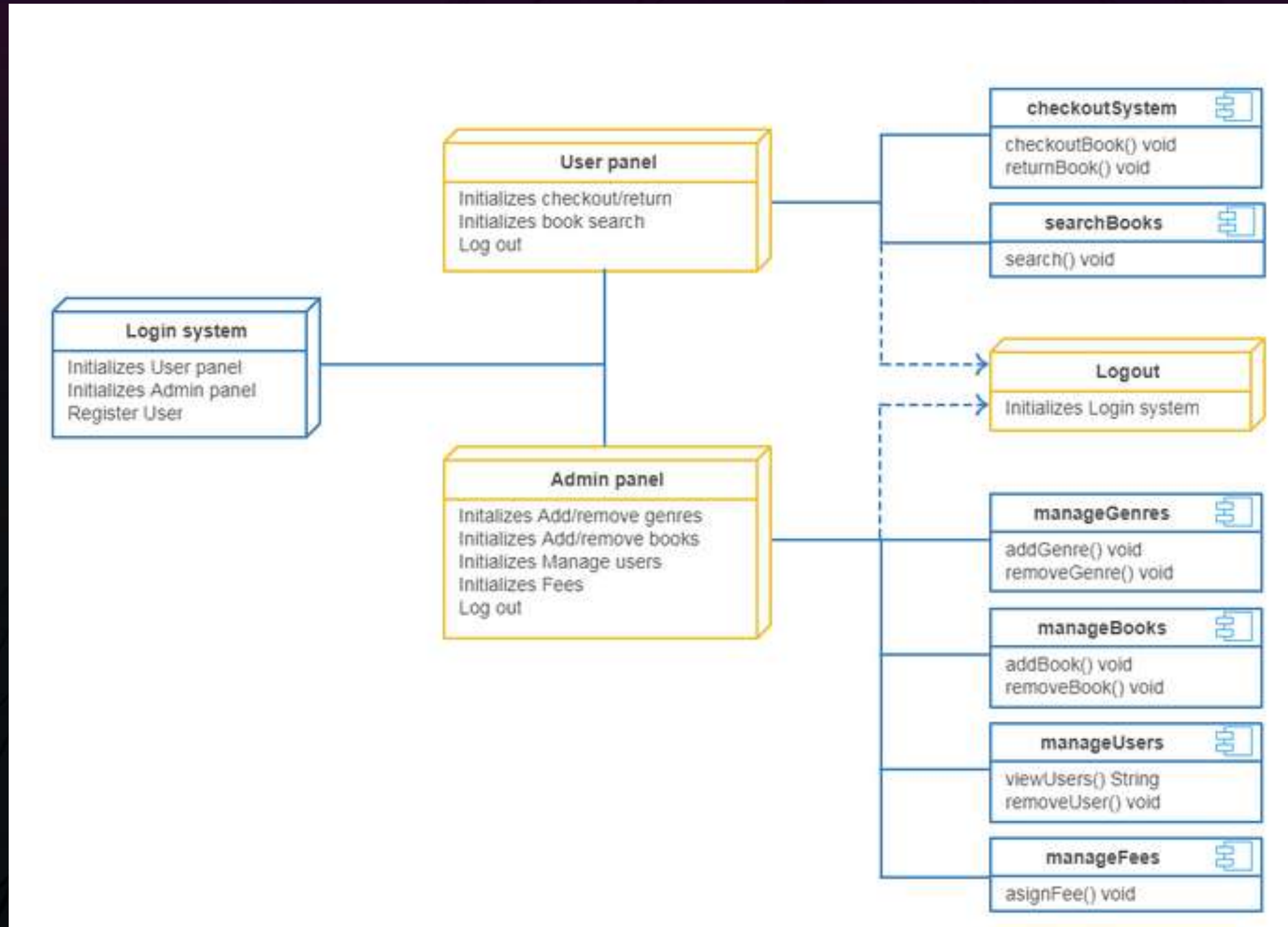


EJEMPLOS

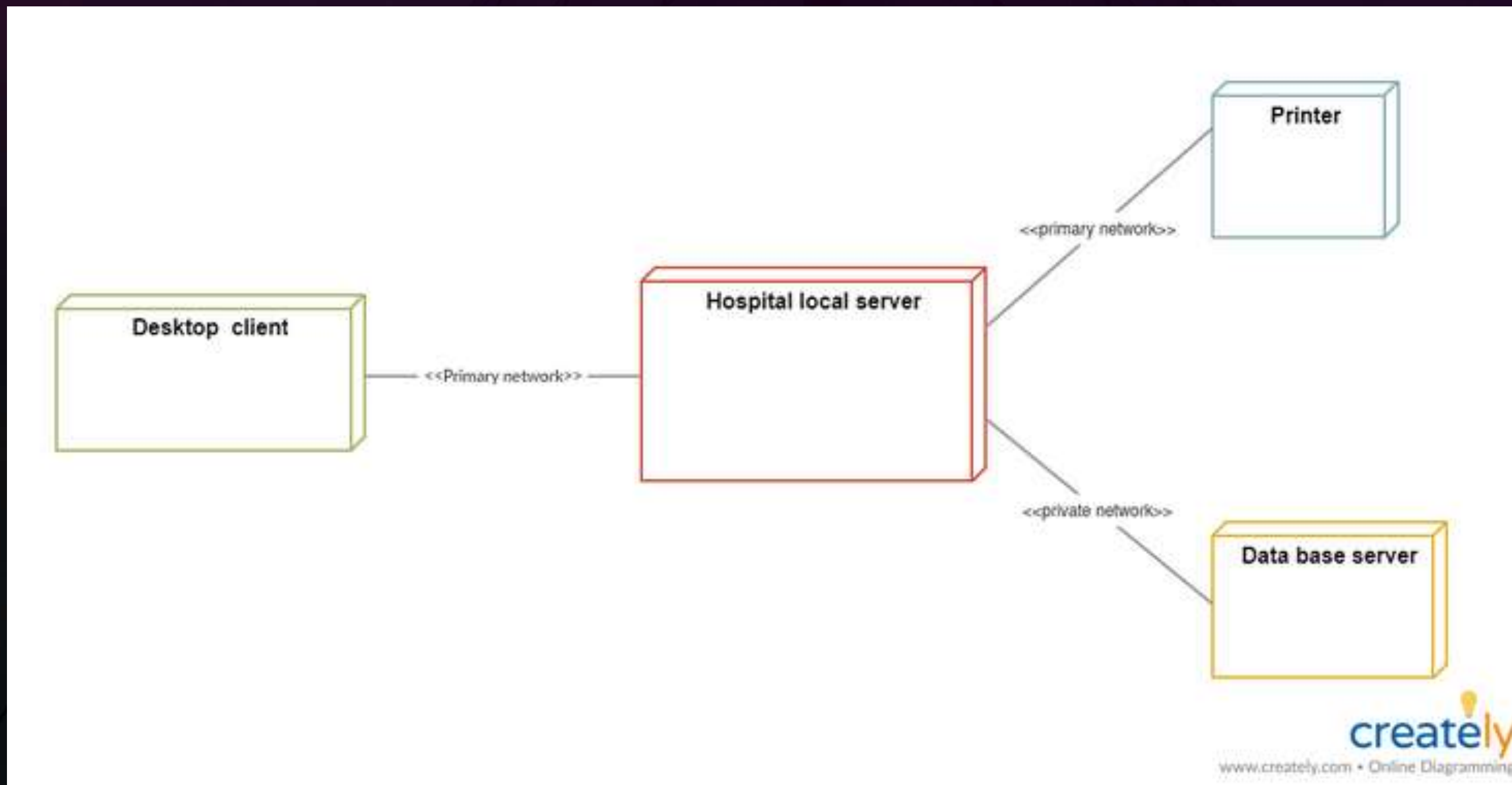
EJEMPLOS - SISTEMA DE GESTION DE COMPRAS EN LINEA



EJEMPLOS - SISTEMA DE GESTION DE UNA BIBLIOTECA



EJEMPLOS - SISTEMA DE GESTION DE UN HOSPITAL





**CUAL SERIA EL QUE MAS
SE PARECE A NUESTRA
API DEL RESTAURANTE
AHORITA?**

ES HORA DE PASAR DE LA TEORIA A LA PRACTICA

- Cada uno en la carpeta de Material de Clase/CORTE 2/Semana 10 Tienen 3 guias Turtwig:
 - Guia Seguridad
 - Guia Dockerizacion
 - Guia CI/CD

La idea es que en este espacio implementen a su ritmo cada una de ellas en su api del restaurante



TAREA

1. Deben terminar la implementacion de la guia para seguridad en sus APIS del restaurante.
2. Deben terminar la implementacion de la guia para contenedores, docker, CI/CD y despliegue en sus APIS del restaurante.
3. Organicen sus README explicando cada diagrama, teniendo un indice de la informacion y mostrandola profesionalmente

FECHA LIMITE: 12 DE OCTUBRE

Recuerden para esa fecha tambien tener sus ejercicios de refuerzo grupo chimchar y piplup



TAREA

1. Todos tienen sus Mocks del restaurante del laboratorio #3 de CORTE 1 – su misión para la semana 11 es que lleguen con un prototipo funcional que cumpla con lo que diseñaron en su API

FECHA LIMITE: 15 DE OCTUBRE



NOS VEMOS EN



LA SESION DE PROYECTO DEL JUEVES